

MorphNet: Complete System Onboarding Guide

MorphNet transforms browser automation (computer use) into reusable API tools (MCP). It observes successful browser interactions, captures HTTP traffic, and crystallizes patterns into deterministic MCP tool calls. Over time, computer use gets replaced by fast API-level execution.

1. System Architecture Overview

MorphNet is a pipeline:

`session_manager` → `orchestrator` → (`CU` | `MCP executor`) → `reflector` → `tracer`

Each iteration of the orchestrator loop:

- 1. **Gets page state** on-demand (AXTree, DOM, screenshots)
- 2. **Builds representations** tailored to each consumer
- 3. **Plans** the next subtask using a branch/prune tree (AgentOccam)
- 4. **Routes** to computer use (CU) for discovery or existing tool graphs for execution
- 5. **Reflects** on the outcome (deterministic checks first, LLM only when ambiguous)
- 6. **Updates** the planning tree and loops

During CU execution, the **observer** captures HTTP traffic, the **noise filter** strips analytics/ads, and the **learner** crystallizes traffic patterns into reusable tool graphs stored in the **manifest**.

Models Used

Component	Model	Temperature	Thinking Budget
Orchestrator (planning)	gemini-3-flash-preview	0.4	2048
CU actions	gemini-3-flash-preview	0.2	2048
Subtask reflection	gemini-3.1-pro-preview	0.2	4096
Parameter classification (learner)	gemini-3-flash-preview	0.1	2048
Intent extraction	gemini-3-flash-preview	0.1	1024

Directory Structure

`morphnet/`
├─ `session_manager.py` # Browser layer – Chrome, CDP, state captur

```

|— morphnet_orchestrator.py # Planning tree, routing, main loop
|— representation.py      # AXTree → TOON views for each consumer
|— computer_use.py        # Browser action agent (10 actions/subtask)
|— reflector.py           # 3-stage verification (deterministic → di
|— observer.py            # HTTP traffic capture via CDP
|— noise_filter.py         # Brave adblock + domain lists
|— learner.py             # 12-step tool discovery pipeline
|— manifest.py            # Tool registry, graph identity, lifecycle
|— executor.py            # Graph execution engine (CDP-based)
|— prompts/               # All LLM prompt templates (.txt)
|   |— orchestrator_plan.txt
|   |— cu_action.txt
|   |— cu_core.txt
|   |— cu_plan.txt
|   |— cu_context_form.txt
|   |— cu_context_search.txt
|   |— cu_context_listing.txt
|   |— cu_context_recovery.txt
|   |— reflect_action.txt
|   |— reflect_subtask.txt
|   |— intent_extraction.txt
|   |— tool_naming.txt
|   |— classify_params.txt
|   |— array_selection.txt
|   |— verification_task_gen.txt
|   |— param_generation.txt
|— sites/                 # Per-site learned state
|   |— confirmkt_com/
|       |— tools.json      # Discovered MCP tool graphs
|       |— profile.json    # Learned website insights
|       |— credentials.json # Login credentials (untracked)

```

2. Session Manager — The Browser Layer

`session_manager.py` (~3200 lines) is the infrastructure foundation. It owns the persistent Chrome browser session and provides raw data to every other module. It contains **zero task logic** — just browser management, state extraction, and action execution.

2.1 Chrome Session Launch

MorphNet connects to Chrome via the **Chrome DevTools Protocol (CDP)** through Playwright:

```

# Connection (not direct WebDriver)
self._playwright = await async_playwright().start()
self._browser = await self._playwright.chromium.connect_over_cdp(self.c
self._context = self._browser.contexts[0]
self.page = await self._context.new_page()

```

Chrome itself is launched as a separate process with stealth flags:

```
chrome \
```

```

--remote-debugging-port=9222 \
--user-data-dir=/tmp/morphnet_profile_9222 \
--no-first-run --no-default-browser-check \
--disable-default-apps --disable-component-update \
--disable-breakpad --disable-sync \
--disable-features=Translate,OptimizationHints,MediaRouter \
# Bot-detection evasion
--disable-blink-features=AutomationControlled \
--exclude-switches=enable-automation \
# Realistic rendering (GPU enabled - disabling is a bot signal)
--use-gl=angle --use-angle=default \
--window-size=1920,1080 \
--force-color-profile=srgb --lang=en-US \
--headless=new # Modern headless (old headless is detectable)

```

Key decisions:

- GPU acceleration is **intentionally enabled** (`--use-gl=angle`). Modern bot detection flags `--disable-gpu`.
- `--headless=new` uses Chrome's modern headless mode (indistinguishable from headed in most fingerprint checks).
- Each instance gets a persistent profile directory for cookie/session reuse.

Default viewport: **1440 × 900 pixels**.

2.2 Bot Detection Avoidance

MorphNet uses a **three-layer stealth strategy**. Despite this, some sites still detect the bot, so experiments typically run in headed mode.

Layer 1: Playwright-Stealth Library

```

stealth = Stealth(
    navigator_languages_override=("en-US", "en"),
    navigator_vendor_override="Google Inc.",
    init_scripts_only=False,
)
await stealth.apply_stealth_async(self._context)

```

Covers ~30 standard fingerprint vectors: `navigator.webdriver`, Chrome properties, `window.chrome`, etc.

Layer 2: Custom Stealth Script

Additional modern detection signals injected via `page.add_init_script()`:

Signal	What It Does	Why It Matters
<code>navigator.connection</code>	Spoofs NetworkInformation API	Cloudflare checks <code>effectiveType</code> , <code>rtt</code> , <code>downlink</code>

<code>chrome.loadTimes()</code>	Adds realistic page load timings	Real Chrome has this; automation frameworks don't
<code>chrome.csi()</code>	Adds Chrome-specific timing API	Same as above
WebRTC IP filtering	Removes private IPs from ICE candidates	Prevents local IP leakage via WebRTC
<code>Intl.DateTimeFormat</code>	Ensures <code>calendar='gregory'</code>	Headless Chrome sometimes reports unexpected locales
<code>MediaDevices</code>	Spoofs audio/video devices	Empty device list is a bot signal
<code>history.pushState</code> <code>capture</code>	Records SPA navigations via <code>Symbol.for('_mn_nav')</code>	Observer uses this to track SPA route changes
Error stack cleaning	Removes <code>__pwInitScripts</code> , <code>__playwright</code> , <code>cdc_markers</code>	Stack traces are fingerprinted by anti-bot systems

Layer 3: Real User-Agent Matching

```
ua = await self.page.evaluate("() => navigator.userAgent")
# Extract Chrome version, match curl_cffi TLS fingerprint
chrome_major = int(re.search(r"Chrome/(\d+)", ua).group(1))
self.http_session = cffi_requests.Session(impersonate=f"chrome{chrome_m
```

The `curl_cffi` session matches Chrome's exact TLS/JA3 fingerprint for HTTP replay outside the browser.

Critical constraint: CDP domains (Runtime, Debugger, Network) are **not** eagerly enabled. Bot protection systems like Akamai and PerimeterX detect `Debugger.enable` artifacts and 403-block all subsequent API calls. CDP is enabled **on-demand per session**.

2.3 State Capture Pipelines

Session manager provides five on-demand state extraction pipelines. Each consumer calls exactly what it needs — no bundled extractions.

2.3.1 AXTree (Accessibility Tree)

The AXTree is the primary page representation. It's fetched directly from Chrome's accessibility engine via CDP:

```
cdp = await self._context.new_cdp_session(self.page)
result = await cdp.send("Accessibility.getFullAXTree")
await cdp.detach()
return self._cdp_axtree_to_nested(result)
```

Raw CDP Response (what Chrome returns):

```
{
  "nodes": [
```

```

{
  "nodeId": "1",
  "role": {"value": "RootWebArea"},
  "name": {"value": "ConfirmTkt - Train Booking"},
  "properties": [
    {"name": "focusable", "value": {"value": true}}
  ],
  "childIds": ["2", "3", "4"],
  "ignored": false
},
{
  "nodeId": "2",
  "role": {"value": "banner"},
  "name": {"value": ""},
  "childIds": ["5", "6", "7"],
  "ignored": false
},
{
  "nodeId": "5",
  "role": {"value": "link"},
  "name": {"value": "Home"},
  "childIds": [],
  "ignored": false
}
]
}

```

Processing: The flat node array is converted to a nested tree. Ignored nodes (wrapper divs with "ignored": true) are made transparent — their children are promoted to the parent level.

After processing (nested structure):

```

{
  "role": "RootWebArea",
  "name": "ConfirmTkt - Train Booking",
  "children": [
    {
      "role": "banner",
      "name": "",
      "children": [
        {"role": "link", "name": "Home", "children": []},
        {"role": "link", "name": "Trains", "children": []},
        {"role": "button", "name": "Search", "children": []}
      ]
    }
  ]
}

```

2.3.2 DOM Tree

Raw HTML fetched via Playwright, then stripped server-side for speed:

```

raw_html = await self.page.content() # Full page HTML

# Python-side regex stripping (~150ms even for multi-MB pages)
cleaned = re.sub(r'<(script|style|noscript|svg|link\s)[^>]*>.*?</\1>',
cleaned = re.sub(r'\s+style="[^"]*"', '', cleaned) # Remove inline s
cleaned = re.sub(r'\s+class="[^"]*"', '', cleaned) # Remove class at
cleaned = re.sub(r'\n\s*\n+', '\n', cleaned) # Collapse white

```

Raw HTML (before):

```

<div class="sc-hKMtZM iNYBPx" style="display:flex;padding:16px">
  <script>window.__DATA__={...}</script>
  <h2 class="css-1a2b3c">Train Search</h2>
  <input type="text" name="from" placeholder="From Station"
    class="styled-input-abc" style="border:1px solid #ccc">
  <style>.autocomplete{position:absolute}</style>
</div>

```

After stripping (cleaned DOM):

```

<div>
  <h2>Train Search</h2>
  <input type="text" name="from" placeholder="From Station">
</div>

```

Truncation: 200KB max. If exceeded, truncated with <!-- DOM truncated -->.

2.3.3 HTTP Traffic

Network monitoring hooks into Playwright's response events:

```

async def _on_response(response: Response):
    request = response.request
    if request.resource_type not in ("xhr", "fetch"):
        return # Only XHR and fetch
    if self._is_noise_url(request.url):
        return # Filter ads/analytics

    # Capture full request/response pair
    captured = CapturedRequest(
        url=request.url,
        method=request.method,
        request_headers=await request.all_headers(),
        response_headers=await response.all_headers(),
        request_body=request.post_data,
        response_body=await response.text(),
        status_code=response.status,
        resource_type=request.resource_type,
        timestamp=time.time(),
    )
    captured.classify_request() # Determine protocol type
    self._captured_traffic.append(captured)

```

Only captures xhr and fetch resource types — documents, images, fonts are ignored.

Example captured request:

```
CapturedRequest (
  url="https://www.confirmkt.com/api/trains/search",
  method="POST",
  request_headers={"content-type": "application/json", "cookie":
  request_body='{"from": "PUNE", "to": "MUMBAI", "date": "2026-(
  response_body='{"trains": [{"number": "12127", "name": "Pune :
  status_code=200,
  protocol="rest",
  endpoint_identity="POST /api/trains/search",
  is_state_changing=True,
)
```

2.3.4 Screenshots

```
async def take_screenshot(self) -> Screenshot:
    page_height = await self.page.evaluate("document.documentElement.sc
    # Pages > 5x viewport height → viewport-only (Gemini rejects huge i
    use_full_page = page_height <= self.viewport_height * 5
    raw = await self.page.screenshot(full_page=use_full_page, type="jpe

    return Screenshot(
        image_base64=base64.b64encode(raw).decode(),
        url=self.page.url,
        viewport_height=dimensions["viewportHeight"],
        full_page_height=dimensions["fullPageHeight"],
    )
```

Screenshot history capped at 20 (reduced to 10 when exceeded) to prevent memory bloat.

2.3.5 Cookies & Token Extraction

```
# Browser cookies
cookies = await self._context.cookies()

# localStorage and sessionStorage
storage = await self.page.evaluate("""() => {
    const local = {}; const session = {};
    for (let i = 0; i < localStorage.length; i++) {
        const k = localStorage.key(i); local[k] = localStorage.getItem(
    }
    // ... same for sessionStorage
    return {local_storage: local, session_storage: session};
}""")
```

Meta token extraction also probes:

- Meta tags: csrf-token, _csrf, authenticity_token
- Hidden form fields: _token, csrfmiddlewaretoken, wp_nonce
- JS variables: window.csrfToken, window.__INITIAL_STATE__
- Storage keys: auth_token, jwt, id_token

Cookies are synced to the `curl_cffi` HTTP session so tool execution outside the browser maintains the same authentication state.

2.4 The `call_gemini()` Utility

All LLM calls across MorphNet go through this single utility in `session_manager.py`:

```
def call_gemini(
    *,
    model: str,
    contents: list[Any],          # Text strings or image dicts
    response_schema: Any = None, # Enables structured JSON output
    system_instruction: str = None,
    generation_config: dict = None,
    prompt_log_dir: Path = None, # Saves prompts for debugging
) -> Any:
```

Key behaviors:

- **Structured output:** When `response_schema` is provided, sets `response_mime_type = "application/json"` and the Gemini API returns validated JSON matching the schema.
- **Image handling:** Converts `{"mime_type": "image/jpeg", "data": "<base64>"}` dicts to the SDK's `Part(inline_data=Blob(...))` format.
- **Retry strategy:** 3 retries with exponential backoff (1s, 2s, 4s) for transient errors. On image processing failure, strips images and retries text-only.
- **Token logging:** Logs prompt/output/thinking token counts for cost tracking.
- **Truncated JSON recovery:** If structured output parsing fails (truncated), retries with `max_output_tokens=16384`.

2.5 Action Execution

Session manager executes browser actions with human-like timing:

Action	Method	Human-Like Behavior
click	Scroll into view → hover (150-500ms pause) → click (40-120ms delay)	Random delays mimicking human motor control
type	Triple-click to select all → character-by-character typing	Bigram speed-up (30%), occasional thinking pauses (300-600ms), burst typing (30-50ms)
scroll	Multiple small increments (60-140px per tick)	Decaying speed with 30-80ms pauses between ticks
select	Try by value first, then by label	Two-attempt fallback
navigate	<code>page.goto()</code> with <code>domcontentloaded</code>	30s timeout
press_key	<code>page.keyboard.press()</code>	Direct, no delays needed
hover	<code>locator.hover()</code>	5s timeout
go_back	<code>page.go_back()</code>	<code>domcontentloaded</code> wait
wait	<code>asyncio.sleep()</code>	Bounded 100ms-10s

Click fallback chain: normal click → force click (handles overlays). **Type fallback chain:**

click + character-type → locator.fill() → contenteditable JS injection.

After every action, navigation detection runs:

```
if not self._is_same_page(url_before, url_after):
    result.navigation_occurred = True
    result.elements_stale = True
    self._previous_elements = [] # Reset stable element IDs
```

2.6 Site Name & Per-Site State

Site name is derived from the URL hostname:

```
hostname = urlparse(start_url).hostname # "www.confirmktkt.com"
if hostname.startswith("www."):
    hostname = hostname[4:] # "confirmktkt.com"
self.site_name = hostname.replace(".", "_") # "confirmktkt_com"
```

Each site gets a directory under morphnet/sites/{site_name}/ with:

- profile.json — learned website insights (navigation patterns, UI quirks)
 - tools.json — discovered MCP tool graphs with lifecycle status
 - credentials.json — login credentials (untracked by git)
-

3. The Orchestrator — Planning & Routing

morphnet_orchestrator.py (~1300 lines) implements the main task loop. It decomposes tasks into subtasks using an AgentOccam-inspired branch/prune planning tree, routes each subtask to either computer use or a learned tool graph, and uses reflector feedback to update the tree.

3.1 The Planning Tree (AgentOccam)

The planning tree is based on AgentOccam's paper, which uses a **branch and prune** approach for task decomposition. Instead of a flat list of steps, the orchestrator maintains a tree where each branch represents an approach to a sub-problem.

Data Structures

```
@dataclass
class PlanNode:
    node_id: str # Hierarchical: "plan_0", "plan_0_1", "pl
    parent_id: str | None # None for root
    intent: str # "Search for trains from Pune to Mumbai"
    status: str # "active" | "completed" | "pruned"
    prune_reason: str | None # "No search results found"
    summary: BranchSummary | None
    children: list[str] # Child node IDs

@dataclass
class BranchSummary:
```

```

what_was_attempted: str      # "Searched for trains using the search b
key_actions: list[str]       # ["Typed 'Pune' in from field", "Clicked
outcome: str                 # "Found 15 trains but no direct routes"
reasoning: str               # "Site requires date selection before se
insights_gained: str         # "Left sidebar has date picker, mandator
data_collected: str | None  # "Cheapest: Pune Intercity at ₹350"

```

How node IDs work: Root is "plan_0". Children append _N: "plan_0_1", "plan_0_2". Grandchildren: "plan_0_1_1", etc. This creates a readable hierarchy.

Tree Operations

branch(intent) — Create a new child under the current node and switch focus to it:

```

def branch(self, intent: str) -> str:
    idx = self._next_child_idx.get(self._current_id, 1)
    new_id = f"{self._current_id}_{idx}"
    self._nodes[new_id] = PlanNode(node_id=new_id, parent_id=self._curr
                                intent=intent, status="active", ...
    self._current_id = new_id  # Focus shifts to new branch
    return new_id

```

prune(reason, summary) — Mark current branch as failed and return to parent:

```

def prune(self, reason: str, summary: BranchSummary | None = None) -> s
    node = self._nodes[self._current_id]
    node.status = "pruned"
    node.prune_reason = reason
    self._current_id = node.parent_id  # Return to parent
    # Safety: if root was pruned, auto-create recovery branch
    if self._current_id is None:
        self._current_id = "plan_0"
        self.branch("Recovery after root prune")

```

complete_current(summary) — Mark current branch as done and return to parent:

```

def complete_current(self, summary: BranchSummary) -> str:
    node = self._nodes[self._current_id]
    node.status = "completed"
    node.summary = summary
    self._current_id = node.parent_id  # Return to parent

```

Visualization: How the Tree Grows and Prunes

Example: Booking a train on ConfirmTkt

Task: "Find the cheapest train from Pune to Mumbai on May 1st"

Step 1 – Initial state:

<pre> plan_0: "Find cheapest train Pune→Mumbai" └─ plan_0_1: "Initial approach" ← CURRENT </pre>
--

Step 2 – After first subtask (search attempted, failed – date required)

```
plan_0: "Find cheapest train Pune→Mumbai"
├─ plan_0_1: "Initial approach" [pruned]
│   Attempted: Searched using search bar without date
│   Outcome: Form validation error – date field required
│   Insights: Date picker is mandatory before search
└─ plan_0_2: "Fill date first, then search" ← CURRENT
```

Step 3 – After second subtask (search succeeded):

```
plan_0: "Find cheapest train Pune→Mumbai"
├─ plan_0_1: "Initial approach" [pruned]
│   Pruned: Form requires date before search
├─ plan_0_2: "Fill date first, then search" [completed]
│   Attempted: Set date to May 1, filled Pune→Mumbai
│   Outcome: Found 15 trains listed
│   Data: Results showing, prices range ₹350–₹1200
└─ plan_0_3: "Sort by price to find cheapest" ← CURRENT
```

Step 4 – Task complete:

```
plan_0: "Find cheapest train Pune→Mumbai"
├─ plan_0_1: [pruned] – date required
├─ plan_0_2: [completed] – search found 15 trains
└─ plan_0_3: [completed] – identified cheapest
    Data: "Pune Intercity (12127) at ₹350, departs 06:15"
```

The Mermaid visualization (saved to `planning_tree.mermaid`) uses colors:

- Green (#90EE90) = completed
- Pink (#FFB6C1) = pruned
- Blue (#87CEEB) = active

3.2 Loop Detection

When the planner keeps trying similar approaches, loop detection kicks in:

```
def detect_repeated_approaches(self) -> str | None:
    # Collect all pruned branch summaries
    pruned_attempts = [node.summary.what_was_attempted.lower()
                        for node in self._nodes.values()
                        if node.status == "pruned" and node.summary]

    # Cluster by 40% word overlap
    for i, a in enumerate(pruned_attempts):
        words_a = set(a.split())
        for j, b in enumerate(pruned_attempts):
            words_b = set(b.split())
```

```

overlap = len(words_a & words_b)
if overlap > 0.4 * max(len(words_a), len(words_b)):
    # Same cluster – approaches are too similar

```

If 2+ pruned branches share >40% word overlap, the planner receives:

```

LOOP DETECTED – The following approaches have been tried multiple times
- Tried 3 times: "searched for product using search bar" (and similar

```

You MUST try a FUNDAMENTALLY DIFFERENT approach. Do NOT retry these str

3.3 Website Insights Pipeline

During task execution, the planner generates `website_insights` — observations about how a website works:

```

# Insights accumulated during task
self._website_insights.append(plan["website_insights"])

# Saved to profile.json at task completion
profile["insights"] = existing_insights + new_insights
profile["insights"] = profile["insights"][-20:] # Keep last 20
profile["last_updated"] = time.time()

```

Example `profile.json` for `ConfirmTkt`:

```

{
  "url": "https://www.confirmtk.com",
  "last_updated": 1714387200,
  "insights": [
    "Search requires both source and destination stations",
    "Date picker is mandatory – cannot search without selecting da",
    "Station names use auto-suggest dropdown, must select from lis",
    "Train results page shows PNR status, not just availability",
    "Booking requires login – redirect to login page on 'Book Now'"
  ]
}

```

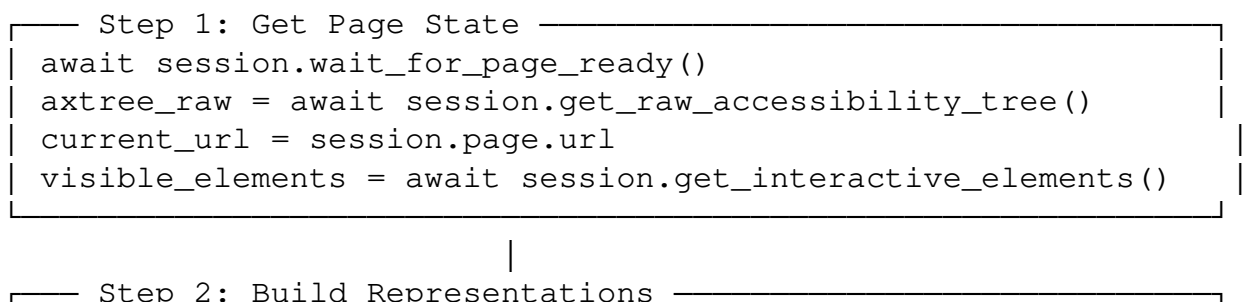
The insights pipeline **appends** to existing insights (not replaces), keeping the most recent 20. This allows the system to learn about a site across multiple task runs.

3.4 The Main Orchestrator Loop

```

for step in range(1, max_subtasks + 1): # Default max_subtasks=15

```



```

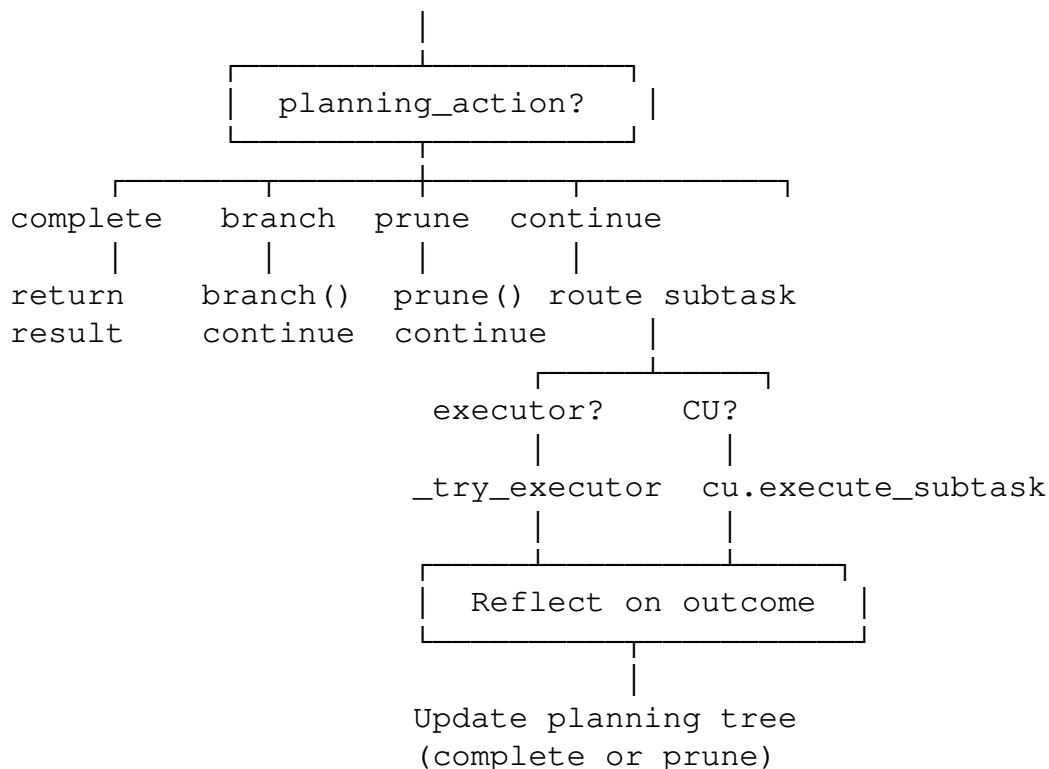
| axtree_view = build_orchestrator_representation(axtree_raw)
| dom_summary = build_dom_summary(dom_tree)
| tree_context = planning_tree.get_context_for_planning()
| loop_warning = planning_tree.detect_repeated_approaches()
| mcp_summary = _get_available_graphs_summary()
| profile_summary = _get_website_profile_summary()
|

```

```

|
| — Step 3: Call Planner LLM —
| plan = await _call_planner(...)
| # Model: gemini-3-flash-preview, temp 0.4
| # Schema: ORCHESTRATOR_SCHEMA
|

```



3.5 Planning Prompt Assembly

The `_call_planner()` function assembles the complete context for the planning LLM:

```

prompt = (
    f"Task: {task_prompt}\n\n"
    f"Current URL: {current_url}\n"
    f"Planning step {step} of {max_subtasks}\n"
    f"{urgency_hint}\n\n"
    f"Current Page (AXTree):\n{axtree_view[:6000]}\n\n"           # Truncate
    f"DOM Summary:\n{dom_summary[:2000]}\n\n"                   # Trunca
    f"Planning Tree:\n{tree_context[:3000]}\n\n"                 # Trunca
    f"Learned Graph Tools:\n{mcp_summary}\n\n"
    f"Website Profile:\n{profile_summary}\n"
)

```

Truncation limits:

Component	Limit	Why
-----------	-------	-----

AXTree view	6,000 chars	Planner needs page overview, not full detail
DOM summary	2,000 chars	Structural signals only
Planning tree	3,000 chars	History + current focus
Executor response	5,000 chars	Previous API results for answer extraction

Additional injections:

- **User credentials** (if available): "User Info (for forms/booking): username: john@ex.com ..."
- **Executor response** (if previous step used API): Injected with instruction to extract answer directly rather than falling back to CU

Urgency hints:

- remaining > 3: no hint
- remaining <= 3: "LOW BUDGET — only {N} subtasks remaining."
- remaining <= 0: "FINAL ACTION — this is your last subtask."

3.6 ORCHESTRATOR_SCHEMA — The Planning Output

Every planner call returns a structured response matching this schema:

Field	Type	Description
assessment	string	2-3 sentence status: what's accomplished, what remains
planning_action	enum	continue / branch / prune / complete_task
branch_intent	string	Intent for new branch (if branching)
prune_reason	string	Why abandon this approach (if pruning)
next_subtask	string	Natural language subtask for CU (must be completable in ≤10 actions)
routing	enum	computer_use / executor — route to CU or learned tool
graph_name	string	Which tool graph to use (if routing = executor)
urgency	enum	normal / low_budget / final_action
website_insights	string	New learnings about this website (saved to profile)
final_answer	string	The task answer (if complete_task)
task_success	boolean	Did the task actually succeed?
reasoning	string	Detailed reasoning for this decision
confidence	number	0.0 to 1.0
evidence_sources	array	What page information informed this decision

3.7 Tool Presentation to the Planner

The `_get_available_graphs_summary()` function presents discovered tools:

```
search_trains [verified]
  Search for trains between two stations on a specific date
  Stats: 15/18 successful executions
  Steps requiring input:
    node_search (Train search API): fromStation, toStation, journeyDa
```

```

check_pnr [probationary]
    Check PNR status for a given PNR number
    Stats: 2/3 successful executions
    Steps requiring input:
        node_pnr (PNR lookup): pnrNumber

```

Lifecycle markers:

- [verified] — tool has proven reliable, safe for reuse
- [probationary] — write-operation tool, needs confirmation before trust
- [unverified] — newly discovered, read-only verification pending

3.8 Response Summary

When the executor returns API data, `_build_response_summary()` formats it for the planner:

1. Takes the **last non-internal node's** output from the execution graph
2. Truncates arrays to 20 items max
3. Caps total JSON at 5,000 characters

This is injected into the next planning call so the planner can extract answers directly from API responses without needing CU to read the page.

3.9 Tool Lifecycle Management

Event	Probationary Graph	Verified Graph
Executor succeeds	Promote → verified	No change
Executor fails	Discard (removed from store)	No change (retry later)
Task ends	Purge all remaining unverified graphs	Preserved

```

# On success: promote probationary to verified
if is_probationary and graph:
    promote_graph(graph.site, graph.id)

```

```

# On failure: discard probationary
if is_probationary and graph:
    discard_graph(graph.site, graph.id)

```

4. Representation — AXTree Views & TOON Notation

`representation.py` (~2500 lines) owns **ALL** AXTree-to-text transformations. Each consumer (planner, CU agent, reflector, executor) gets a tailored view of the same raw AXTree. The key innovation is **TOON (Token-Optimized Object Notation)** — a compact format achieving ~40% token savings over JSON.

4.1 TOON Format

TOON is the compact notation used to represent interactive elements:

```
[ID] role"Name"="Value" state |near:Context
```

Components:

Part	Example	When Included
[ID]	[5]	Always (element identifier for CU)
role	btn, txt, dd, lnk, chk	Always (abbreviated from ARIA role)
"Name"	"ADD", "Search"	If element has a name
"Value"	"Medium", "Pune"	If element has a current value
Ø	txt"Email"Ø	Empty textbox (visual cue)
State flags	✓ (checked), exp (expanded), col (collapsed), dis (disabled), req (required), foc (focused)	If applicable
	near:Vietnamese Cold Brew	For generic/short names needing disambiguation
near:Context		

Role abbreviations (_ROLE_MAP):

ARIA Role	TOON	ARIA Role	TOON
button	btn	textbox	txt
link	lnk	searchbox	search
combobox	dd	checkbox	chk
radio	radio	switch	sw
menuitem	menu	tab	tab
slider	slider	spinbutton	spin

Examples:

[1] txt"From Station"Ø, req	← Empty required text fi
[2] txt"To Station"="MUMBAI CST"	← Filled text field
[3] dd"Travel Class"="Sleeper", col	← Collapsed dropdown
[4] btn"Search Trains"	← Simple button
[5] btn"ADD" near:Vietnamese Cold Brew	← Generic button disamb
[6] chk"Remember me" ✓	← Checked checkbox
[7] lnk"View Details" near:Pune Intercity	← Link needing context

4.2 The Context Stack

The `_ContextStack` class solves the problem of **disambiguating generic buttons**. When you see three "ADD" buttons on a page, which product does each one add?

Mechanism: Depth-keyed, first-text-wins.

```
class _ContextStack:
    def __init__(self):
        self._stack: dict[int, str] = {} # depth → first text at that
```



```

def update(self, depth: int, text: str):
    if depth not in self._stack:          # First text wins
        self._stack[depth] = text
    # Clear all deeper entries
    for d in list(self._stack):
        if d > depth: del self._stack[d]

def get_nearest(self, depth: int, max_lookback: int = 4) -> str | N
    for d in range(depth, max(depth - max_lookback, -1), -1):
        if d in self._stack:
            return self._stack[d][:80]

```

Example walkthrough (product card list):

Depth 2: "Vietnamese Cold Brew"	← First text at depth 2, saved
Depth 3: "Rs 250"	← First text at depth 3, saved
Depth 3: "In Stock"	← Depth 3 already has text, skipped
Depth 3: [ADD button]	← get_nearest(3) → "Vietnamese Cold"
Depth 2: reset_scope(2)	← New sibling card starts, clear dept
Depth 2: "Iced Latte"	← New first text at depth 2
Depth 3: "Rs 180"	← New first text at depth 3
Depth 3: [ADD button]	← get_nearest(3) → "Iced Latte" (dept

Result: [5] btn"ADD" |near:Vietnamese Cold Brew and [8] btn"ADD" |
near:Iced Latte

4.3 Text Deduplication

The `_TextDedup` class suppresses repetitive text that bloats the representation:

```

class _TextDedup:
    def __init__(self, window=3, max_repeats=3):
        self._recent = []          # Sliding window of recent texts
        self._global_counts = {}   # Text → occurrence count

    def is_duplicate(self, text: str) -> bool:
        normalized = normalize_whitespace(text.lower())
        # Global cap: same text > 3 times → always suppress
        if self._global_counts.get(normalized, 0) >= self._max_repeats:
            return True
        # Rolling window: same text in last 3 → suppress
        if normalized in self._recent:
            return True

```

This prevents accessibility noise like "Skip to content" (appearing 32x) or "Learn more" (24x) from flooding the representation.

4.4 Four AXTree Views

Each consumer gets a different view of the same raw AXTree:

4.4.1 CU View (`build_cu_representation`)

For: Computer use agent (browser actions) **Features:** Section-based, inline element IDs, context stack, footer excluded

Example output:

```
heading "Train Search"
  text "Find trains between stations"
  [1] txt"From Station"Ø, req
  [2] txt"To Station"Ø, req
  [3] dd"Date"="Select date", col
  [4] btn"Search Trains"

heading "Popular Routes"
  [5] lnk"Pune → Mumbai" |near:Daily, 15 trains
  [6] lnk"Delhi → Jaipur" |near:Daily, 22 trains
  [7] lnk"Chennai → Bangalore" |near:Daily, 18 trains
  ... and 12 more link items (IDs [8]-[19])

Fields:
  [1] txt"From Station"Ø, req
  [2] txt"To Station"Ø, req
  [3] dd"Date"="Select date", col

Actions:
  [4] btn"Search Trains"
  [5] lnk"Pune → Mumbai"
```

Key processing:

1. **Noise roles pruned:** `inlinetextbox`, `linebreak`, `mark`, `emphasis`, `strong`, `code`, `svg` — node skipped but children preserved
2. **Generic roles transparent:** `none`, `generic`, `group`, `section`, `div`, `span` — pass through without emission
3. **Footer excluded:** Nodes with role `contentinfo` or names containing `"©"`, `"privacy"`, `"terms of service"`
4. **Repetitive group collapse:** 5+ siblings with same role → show first 3, collapse rest: `"... and 12 more link items (IDs [8]-[19])"`
5. **CSS class filtering:** Framework artifacts like `css-1a2b3c`, `sc-bdVTJa`, `styled-xyz` excluded

4.4.2 Orchestrator View (`build_orchestrator_representation`)

For: Planning LLM (high-level decisions) **Features:** Text-only, NO element IDs, NO footer exclusion, actionable summary at bottom

Example output:

```
Page: "ConfirmTkt - Train Booking"

heading "Train Search"
  text "Find trains between stations"
```

```

text "From Station" (empty)
text "To Station" (empty)

heading "Popular Routes"
  text "Pune → Mumbai – Daily, 15 trains"
  text "Delhi → Jaipur – Daily, 22 trains"

Available actions: "Search Trains", "Login", "Help", "Pune → Mumbai"

```

The planner doesn't need element IDs — it plans at the strategy level, not the action level.

4.4.3 Reflector View (`build_reflector_representation`)

For: Subtask verification LLM **Features:** Content-focused, chrome compressed, card-aware

Example output:

```

Page: "Search Results"

[banner: Home, Trains, PNR Status, Login]

heading "15 Trains Found: Pune → Mumbai"
  [5 items]
  • Pune Intercity (12127) | ₹350 | Dep 06:15 | Arr 09:45
  • Deccan Express (11007) | ₹280 | Dep 07:30 | Arr 11:15
  • Shatabdi Express (12029) | ₹650 | Dep 08:00 | Arr 11:00

[contentinfo: About, Contact, Privacy]

```

Navbar and footer are compressed to one-line summaries. Repeating items (product cards, search results) are detected by structural fingerprinting and formatted as compact bullet lists.

4.4.4 MCP View (`build_tool_param_context`)

For: Parameter generation LLM (filling tool graph parameters) **Features:** DOM-focused, extraction recipe execution, current values

Example output:

```

TASK: Search trains from Pune to Mumbai on May 1st
TOOL: search_trains (POST /api/trains/search)
PROTOCOL: REST

LAST SUCCESSFUL REQUEST:
  {"from": "PUNE", "to": "MUMBAI", "date": "2026-04-15"}
  → Response: 200

PARAMETERS:
  from [user_intent]:
    Station code for departure
    Current value: (none)

```

```

to [user_intent]:
    Station code for destination
    Current value: (none)

_csrf [website_generated]:
    Cookie '_csrf'. Changes every session.
    Current value: "xK9m..."

```

CURRENT URL: <https://www.confirmtk.com>

4.5 Noise Roles and Transparent Nodes

During the AXTree walk, certain roles are pruned or made transparent:

Noise roles — node is skipped, but children are preserved (promoted to parent):

```

_NOISE_ROLES = frozenset({
    "inlinetextbox", "linebreak", "labeltext", "paragraph", "mark",
    "svg", "svgroot", "abbr", "superscript", "subscript", "ruby",
    "rubytext", "insertion", "deletion", "emphasis", "strong", "code",
    "time", "pre", "blockquote", "figcaption", "figure", "details",
    "summary",
})

```

Generic/transparent roles — pass through without emission (container only): none, generic, GenericContainer, group, section, div, span

Example: How noise roles work

```

Raw AXTree:
  paragraph
    staticText "Welcome to "
    emphasis
      staticText "ConfirmTkt"
    staticText " – your train booking partner"

```

After noise role pruning (emphasis skipped, children promoted):
 text "Welcome to ConfirmTkt – your train booking partner"

4.6 The build_cu_representation Pipeline (Step by Step)

The CU representation is built via a multi-step pipeline:

1. Filter elements through `should_include_element()`
 ↓ (removes CSS-class artifacts like "sc-bdVTJa", "css-1a2b3c")
2. Build lookups: (role, normalized_name) → elements
 ↓
3. Initialize: `_ContextStack`, `_TextDedup`, `context_map`, `consumed_ids`
 ↓
4. Recursive `_cu_walk()` over AXTree
 ↓ (produces indented section-based output with inline [ID] elemen
5. Merge `enriched_context` for unmatched elements
 ↓

6. Generate Fields/Actionable quick-reference summary

↓

7. Return concatenated lines

The recursive walk handles three node categories:

- **Structural roles** (heading, navigation, banner, main, form): Emit as section headers, update context stack
- **Interactive roles** (button, textbox, link, etc.): Format as TOON inline elements with [ID]
- **Text roles** (staticText, etc.): Emit as plain text, update context stack for nearby disambiguation

4.7 Enrichment System

Four enrichers run before representation building. Each has a **circuit breaker** — disabled after 3 consecutive failures to prevent log flooding on problematic pages:

```
_enricher_failures: dict[str, int] = {}    # enricher name → consecutive
_ENRICHER_MAX_FAILURES = 3
```

```
for enricher in _ENRICHERS:
    if _enricher_failures.get(name, 0) >= _ENRICHER_MAX_FAILURES:
        continue # Circuit breaker: skip this enricher
    try:
        results = await enricher(elements, page, axtree)
        _enricher_failures[name] = 0    # Reset on success
    except Exception:
        _enricher_failures[name] += 1  # Increment; disable at 3
```

Enricher	What It Does	Strategy	Confidence
enrich_label_association	Links unnamed controls to visual labels	Walk 3 levels up, collect sibling text; fall back to next/prev sibling	0.85
enrich_unnamed_elements	Recovers names from DOM signals	Try in order: img[alt] → svg title → data-label/name/ tooltip → data-testid (humanized) → ancestor text (if sole interactive)	0.70
enrich_card_grouping	Groups elements within repeated cards	Fingerprint: classes(sorted) child_tags(filtered). Requires 3+ containers with same fingerprint	0.75

<code>enrich_lazy_loaded_content</code>	Extracts prices/ ratings from data- * attrs	Three strategies: (1) data-price/ amount/cost attrs, (2) currency regex scan on visible leaf elements, (3) rating attrs. Associates to nearest interactive element (<200px)	0.60
---	--	--	------

Example: `enrich_unnamed_elements` recovery chain

Unnamed button with `<img alt="Close">` inside
 → Strategy 1 (`img alt`): ✓ name = "Close"

Unnamed button with `data-testid="btn-add-to-cart"`
 → Strategy 4 (`data-testid`): ✓ name = "add to cart"
 (humanized: remove "btn-" prefix, replace hyphens with spaces)

Unnamed link, sole interactive inside `<div>Quick View</div>`
 → Strategy 5 (`ancestor text`): ✓ name = "Quick View"

4.8 Repetitive Group Collapse

When 5+ consecutive siblings share the same role, the representation collapses them:

Before collapse (20 product cards):

```
[1] btn"Vietnamese Cold Brew" ...
[2] btn"Iced Latte" ...
[3] btn"Flat White" ...
[4] btn"Cappuccino" ...
... (16 more)
```

After collapse:

```
[1] btn"Vietnamese Cold Brew" ...
[2] btn"Iced Latte" ...
[3] btn"Flat White" ...
... and 17 more button items (IDs [4]-[20])
```

Exception: Groups with rich interactive content (buttons, links, forms inside items) are preserved in full — they represent actionable product cards, not repetitive navigation items.

4.9 DOM Summary

The DOM tree undergoes a separate processing pipeline for the planner:

1. **Strip tags:** `<script>`, `<style>`, `<noscript>`, `<svg>`, `<link>` removed entirely
2. **Strip attributes:** `class="..."` and `style="..."` removed
3. **Collapse whitespace:** Multiple newlines → single newline
4. **Truncation:** 200KB max

The result is a lightweight HTML skeleton showing document structure without noise.

5. Computer Use Agent — Browser Actions

`computer_use.py` (~1400 lines) is the browser action agent. It executes subtasks by taking up to **10 actions** per subtask, using the CU-specific AXTree representation with element IDs.

5.1 Action Types

The CU agent can perform these actions:

Action	Required Fields	Description
click	element_id	Click an interactive element
type	element_id, text	Type text into a field (optional <code>clear_first</code>)
select	element_id, value	Choose dropdown option
scroll	direction, scroll_amount	Scroll page up/down
press_key	text (key name)	Send keyboard key (Enter, Tab, Escape, etc.)
navigate	text (URL)	Go to URL
hover	element_id	Hover over element
go_back	—	Browser back button
wait	—	Pause for page updates
note	text	Record observation (no browser action)
stop	text (reason)	Declare subtask complete or impossible

Every action response includes reasoning, confidence (0.0-1.0), and `evidence_sources` (which page elements informed the decision).

5.2 The Action Loop

```
for step in range(1, 11): # Max 10 actions per subtask

    # 1. Human-like pause (0.8-2.0 seconds)
    await asyncio.sleep(0.8 + random.random() * 1.2)

    # 2. Build pruned AXTree
    axtree = build_cu_representation(raw_axtree, interactive_elements)

    # 3. Maybe take SoM screenshot
    # (always on step 1, after failures, after 2+ consecutive failures)
    if step == 1 or last_action_failed or consecutive_failures >= 2:
        screenshot = await generate_som_screenshot(...)

    # 4. Build action history (last 2 full, older one-liners)
    history = build_action_history(records, step)

    # 5. Try batch action (form fill or search) OR single action
    if not batch_disabled and page_has_forms_or_search:
        batch_result = await _try_batch_action(...)
    if not batch_result:
        action = await _select_action(subtask, axtree, history, step, s

    # 6. Execute action via session_manager
```

```

result = await session.execute_action(action)

# 7. Reflect on action outcome
reflection = await reflector.reflect_on_action(action, result, ...)

# 8. Track failures
if not reflection.success:
    consecutive_failures += 1

```

5.3 The Prompt (What the LLM Sees)

Subtask: Fill in the train search form with Pune to Mumbai on May 1st

Step 3 of 10 (7 remaining)

Current Page:

heading "Train Search"

text "Book your train tickets"

[1] txt"From Station"="PUNE"

[2] txt"To Station"Ø, req

[3] dd"Date"="Select date", col

[4] btn"Search Trains"

[5] lnk"Popular Routes"

heading "Why ConfirmTkt?"

text "98% accuracy in predictions"

... and 5 more listitem items

Fields:

[1] txt"From Station"="PUNE"

[2] txt"To Station"Ø, req

[3] dd"Date"="Select date", col

Actions:

[4] btn"Search Trains"

[5] lnk"Popular Routes"

Action History:

Step 1: Typed 'PUNE' into [1] – success (From Station populated)

Step 2 (full detail):

Action: type [2] "MUMBAI"

Reasoning: Need to fill destination station

Result: success

Changed: To Station field now shows 'MUMBAI'

AXTree truncation: 12,000 characters in single-action mode, 10,000 in batch mode.

5.4 Batch Actions (Plan-Then-Execute)

For pages with forms or search fields, the CU agent can plan and execute multiple actions at once:

Phase 1 — Plan (uses CU_PLAN_SCHEMA):

```
{
  "action_type": "fill_form",
  "target_description": "Fill the train search form with Pune to Mumbai",
  "reasoning": "Form has 3 fields: [1] from, [2] to, [3] date. All need",
  "confidence": 0.9
}
```

Phase 2 — Execute (uses FILL_FORM_SCHEMA or SEARCH_AND_SELECT_SCHEMA):

```
{
  "field_actions": [
    {"element_id": 1, "action": "type", "value": "PUNE", "clear_first":
    {"element_id": 2, "action": "type", "value": "MUMBAI", "clear_first":
    {"element_id": 3, "action": "click", "value": ""}
  ],
  "submit_action": {"element_id": 4, "action": "click"}
}
```

For search-and-select (autocomplete):

```
{
  "search_field_id": 1,
  "query": "Pune",
  "suggestion_preference": "Pune Junction",
  "submit_after_select": false
}
```

Safety: Batch actions only execute on native form elements (<input>, <textarea>, <select>). Custom widget form fields trigger abort — the system falls back to single-action mode.

5.5 Failure Handling

Condition	Response
First failure	Screenshot taken on next step
2+ consecutive failures	Screenshot always included, loop detection active
Same action on same element fails 3+ times	Warning injected: "Action X on element Y has failed N times"
2 consecutive batch failures	Batch mode disabled for rest of subtask
Action loop detected (3+ repetitions in last 6 actions)	Warning: "STOP: you are repeating the same action"

5.6 Contextual Prompt Injection (AXI Pattern)

Instead of one monolithic prompt, CU uses **modular contextual injection** — a small core prompt plus page-context-aware extensions:

cu_core.txt (always loaded, ~600 tokens)

```

+ cu_context_form.txt      (if page has forms)
+ cu_context_search.txt    (if page has search fields)
+ cu_context_listing.txt   (if page has item listings)
+ cu_context_recovery.txt  (if last action failed)

```

How page context is detected:

```

def _build_contextual_prompt(elements, last_action_failed):
    parts = [cu_core_prompt]

    context = analyze_page_context(elements)  # Detects has_form, has_s

    if context.has_form:      parts.append(cu_context_form)
    if context.has_search:    parts.append(cu_context_search)
    if context.has_listing:   parts.append(cu_context_listing)
    if last_action_failed:    parts.append(cu_context_recovery)

    return "\n\n".join(parts)

```

What each context injection teaches the LLM:

Context	Key Instructions
form	Type text as a SEPARATE action from submission. Check AXTree after typing to verify values. Use <code>select</code> for dropdowns (not <code>type</code>). Wait for autocomplete suggestions after typing, then click the correct option.
search	Type query → wait for suggestions → click suggestion OR press Enter. Never combine typing and pressing Enter in the same action.
listing	Use <code>note</code> to record items (name, price, rating). Scroll to reveal more. If 2 scroll attempts reveal no new content, all items are loaded. Don't click into items unless detail isn't on the card — AXTree shows card content.
recovery	Click a DIFFERENT element than the one that failed. Try different page section (sidebar, filters, footer links). Use <code>navigate</code> if URL pattern is inferrable. DO NOT retry the same failing action with minor variations.

5.7 SoM (Set-of-Mark) Screenshot Generation

SoM overlays element ID labels directly onto the page screenshot so the LLM can visually correlate [5] in the AXTree with the actual button on screen:

```

async def generate_som_screenshot(page, elements, viewport_height):
    # 1. Build overlay data
    elem_data = [{id, x, y, w, h} for each visible element]

    # 2. Inject overlay via JavaScript
    #   - Create __morphnet_som_overlay__ div (position:absolute, z-in
    #   - For each element:
    #       → Thin border: 1px solid rgba(30,80,220,0.45)
    #       → ID pill at top-left: rgba(30,80,220,0.78) background, whit
    #         10px bold monospace

    # 3. Take screenshot (JPEG q85, viewport-only if page > 5x viewport

```

```
screenshot = await page.screenshot(full_page=use_full_page, type="j

# 4. Remove overlay
await page.evaluate("document.getElementById('__morphnet_som_overla

return base64.b64encode(screenshot)
```

What the LLM sees: A screenshot where each interactive element has a small blue pill showing [1], [2], [3] etc., with thin borders around clickable/typeable areas. This visual grounding dramatically reduces element misidentification.

5.8 Observer Integration

During CU execution, the observer records every action for later tool discovery:

```
# Before executing each action, CU records it with the observer:
if self._observer:
    target_info = await session._resolve_element(element_id)
    await self._observer.record_cu_action(
        action_type=action["action_type"],
        target={
            "element_id": element_id,
            "selector": target_info.selector,
            "text": target_info.text,
            "ax_node_id": target_info.ax_node_id,
            "attributes": target_info.attributes,
        },
        value=action.get("text", "")[:500],
        reasoning=action["reasoning"],
    )
```

The observer stores each action as a `CUAction` with timestamp, subtask ID, and all target details. After the subtask completes, the learner correlates these timestamps with captured HTTP traffic to build action→API mappings.

5.9 Page Crash Recovery

CU includes multi-level crash recovery:

```
Action fails with "target closed" / "crashed" / "context destroyed"
|
|— Try 1: reattach_page() — reconnect to existing page
|   |— Fails? ↓
|— Try 2: _recover_from_crash(before_url)
|   |— Create new page in existing browser context
|   |— Re-register traffic capture handlers
|   |— Navigate to fallback URL
|   |— Fails? ↓
|— Try 3: Empty AXTree check (< 5 nodes = page still transitioning
|   |— Retry extraction with sleep (up to 3 times)
```

5.10 Screenshot Strategy

Screenshots are **not** taken on every step (tokens are expensive). They are included:

1. **Always** on step 1 (initial page state)
2. **After failures** (what went wrong?)
3. **After 2+ consecutive failures** (every step gets a screenshot)

Screenshots are annotated with **SoM (Set-of-Mark)** overlays — temporary DOM borders with element ID labels — so the LLM can visually correlate [5] in the AXTree with the actual button on screen.

6. Observer — HTTP Traffic & DOM Capture

`observer.py` (~740 lines) monitors everything happening during CU execution. It captures HTTP traffic, DOM snapshots, navigation events, and CU actions — the raw material for the learner to build tool graphs.

6.1 Data Structures

```
@dataclass
class CUAction:
    timestamp: float          # Unix ms
    subtask_id: str
    action_type: str          # click, type, select, scroll, etc.
    target_selector: str | None
    target_attributes: dict
    target_text: str | None
    ax_node_id: str | None
    typed_value: str | None
    reasoning: str

@dataclass
class HTTPRequest:
    url: str
    method: str               # GET, POST, PUT, DELETE
    request_headers: dict
    response_headers: dict
    request_body: str | None
    response_body: str | None
    status_code: int
    classification: str       # "rest", "graphql", "json_rpc", "form"
    initiator_stack: list     # JavaScript call stack
    timestamp: float

@dataclass
class DOMSnapshot:
    timestamp: float
    subtask_id: str
    url: str
```

```

    ax_tree: dict                # Full accessibility tree
    dom_hash: str                # SHA256 of first 50KB of innerHTML
    storage_keys: dict           # Cookie names, localStorage/sessionStorage

@dataclass
class NavigationEvent:
    timestamp: float
    type: str                    # "pushState" or "replaceState"
    url: str

@dataclass
class SubtaskObservation:
    site: str
    task_description: str
    start_url: str
    end_url: str
    cu_actions: list[CUAction]
    http_requests: list[HTTPRequest]
    scripts: dict[str, ScriptSource]
    dom_snapshots: list[DOMSnapshot]
    navigation_events: list[NavigationEvent]
    framework_fingerprint: dict
    bundle_hash: str

```

6.2 CDP Integration

Observer enables four Chrome DevTools Protocol domains:

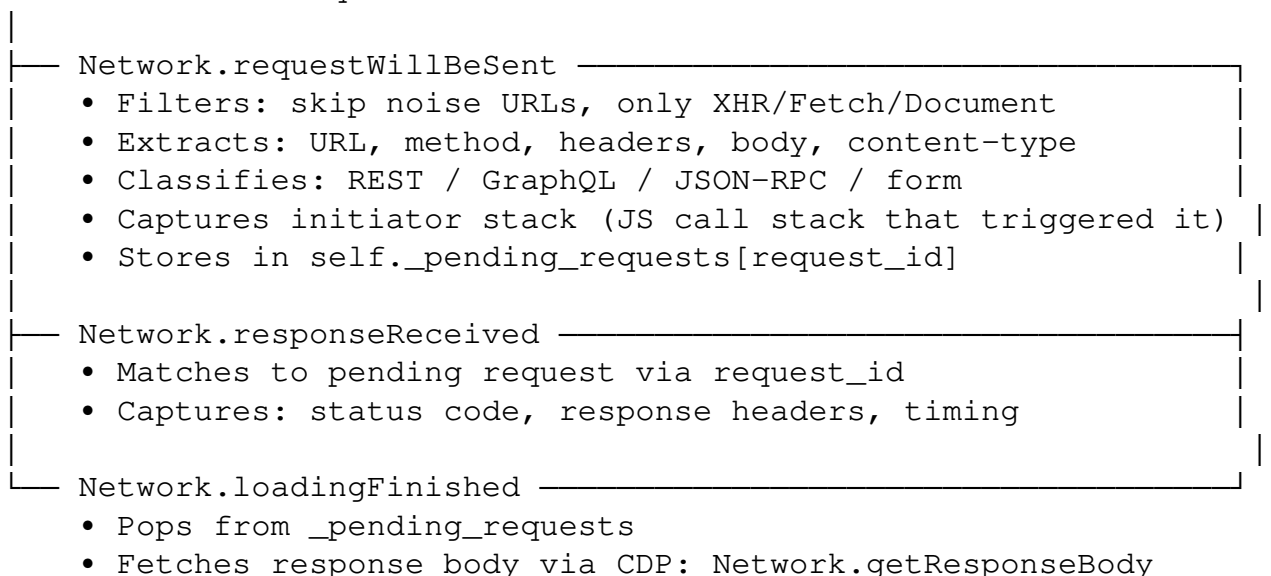
```

await cdp.send("Network.enable")           # HTTP capture
await cdp.send("Runtime.enable")           # JS runtime
await cdp.send("Debugger.enable")          # Script debugging
await cdp.send("Debugger.setAsyncCallStackDepth", {"maxDepth": 32})
await cdp.send("Target.setAutoAttach", {...}) # Service workers/iframe

```

Four event handlers are registered, forming a pipeline:

Browser makes HTTP request



- Skips base64-encoded (binary) responses
- Creates finalized HTTPRequest record
- Captures script sources from initiator stack frames

Debugger.scriptParsed

- Triggered for every JS file loaded
- Fetches source via Debugger.getScriptSource
- Deduplicates by SHA256 content hash

Example: What the observer captures for a single API call

```
requestWillBeSent:
  request_id: "23.45"
  url: "https://www.confirmkt.com/api/trains/search"
  method: "POST"
  body: '{"from": "PUNE", "to": "MUMBAI", "date": "2026-05-01"}'
  request_type: "rest"
  initiator_stack: [
    {functionName: "searchTrains", scriptId: "42", lineNumber: 150},
    {functionName: "handleSubmit", scriptId: "42", lineNumber: 89,
  ]

responseReceived:
  status: 200
  headers: {"content-type": "application/json"}

loadingFinished:
  response_body: '{"trains": [{"number": "12127", "name": "Pune Ir
→ Script source for scriptId "42" also captured via Debugger.get
```

The **initiator stack** is critical — the learner uses it to find JavaScript entry points for tool execution (Strategy A: reachable globals, Strategy C: extracted functions).

Difference from WALT and similar systems: Observer uses CDP's event-based model rather than proxy-based interception. This is faster (no MITM overhead), captures WebSocket/streaming responses, and doesn't break HTTPS certificate pinning. The trade-off is that it requires Chrome cooperation rather than being transport-layer agnostic.

6.3 CU Action Recording

The observer records every CU browser action with its context:

```
async def record_cu_action(self, action_type, target, value, reasoning)
  action = CUAction(
    timestamp_ms=int(time.time() * 1000),
    subtask_id=self._subtask_id,
    action_type=action_type,                # "click", "type", "select",
    target_selector=target["selector"], # CSS selector
    target_attributes=target["attributes"],
    target_text=target["text"],
    target_ax_node_id=target["ax_node_id"],
    typed_value=value,                      # What was typed/selected
```

```

        cu_reasoning=reasoning,                # LLM's reasoning for this a
    )
    self._cu_actions.append(action)

```

These recorded actions are later correlated with HTTP traffic by timestamp in the learner's **CU action windowing** step — determining which browser action triggered which API call.

6.4 HTTP Request Classification

Observer classifies every captured HTTP request by examining the body structure:

```

def _classify_request_type(url, method, content_type, body):
    try:
        parsed = json.loads(body)
    except:
        parsed = None

    # 1. GraphQL: body has "query" or "mutation" key
    if isinstance(parsed, dict) and ("query" in parsed or "mutation" in
        operation = parsed.get("operationName")
        query_hash = sha256(parsed["query"][:12])
        return ("graphql", operation, query_hash, None)

    # 2. JSON-RPC: body has "jsonrpc" key
    if isinstance(parsed, dict) and "jsonrpc" in parsed:
        return ("json_rpc", None, None, parsed.get("method"))

    # 3. Form: content-type is form-urlencoded or multipart
    if "form-urlencoded" in content_type or "multipart" in content_type:
        return ("form", None, None, None)

    # 4. Default: REST
    return ("rest", None, None, None)

```

Example: REST request

```

POST /api/trains/search HTTP/1.1
Content-Type: application/json

```

```

{"from": "PUNE", "to": "MUMBAI", "date": "2026-05-01"}

```

```

→ Classification: ("rest", None, None, None)
→ endpoint_identity: "POST /api/trains/search"

```

Example: GraphQL request

```

POST /graphql HTTP/1.1
Content-Type: application/json

```

```

{
  "operationName": "SearchTrains",
  "query": "query SearchTrains($from: String!, $to: String!) { tra

```

```
    "variables": {"from": "PUNE", "to": "MUMBAI"}
  }
```

→ Classification: ("graphql", "SearchTrains", "a3f8b2c1d4e5", None)

Example: JSON-RPC request

```
POST /rpc HTTP/1.1
Content-Type: application/json
```

```
{"jsonrpc": "2.0", "method": "train.search", "params": {"from": "PUNE", "to": "MUMBAI"}}
```

→ Classification: ("json_rpc", None, None, "train.search")

Example: Form request

```
POST /search HTTP/1.1
Content-Type: application/x-www-form-urlencoded
```

```
from=PUNE&to=MUMBAI&date=2026-05-01
```

→ Classification: ("form", None, None, None)

6.5 Navigation Event Collection

SPA (Single Page Application) navigations don't trigger full page loads. Observer captures them via a Symbol-keyed global:

```
// Injected by stealth script
const original = history.pushState;
history.pushState = function(...args) {
  window[Symbol.for('_mn_nav')].push({
    ts: Date.now(),
    type: 'pushState',
    url: args[2] || location.href
  });
  return original.apply(this, args);
};
```

Observer periodically collects from `window[Symbol.for('_mn_nav')]` and clears the array.

6.6 DOM Snapshots

Snapshots are taken at:

1. **Start of task** — baseline
2. **End of each subtask** — track state changes
3. **Every 5 seconds** (periodic background task)
4. **On URL change**

Each snapshot captures:

- Full accessibility tree
- DOM content hash (SHA256 of first 50KB of innerHTML)
- Storage keys (cookie names, localStorage/sessionStorage key names — not values)

The DOM hash enables quick change detection without full tree comparison.

6.7 Observer Lifecycle

```
start_task(site, task_description)
|— Enable CDP domains
|— Framework fingerprinting
|— Initial DOM snapshot
|— Start periodic snapshots (every 5s)

    start_subtask(subtask_id)      ← Called per subtask
    |   |— Record subtask boundary
    |   |— Continue capturing
    |   |
    |   ... CU actions happen, HTTP traffic flows ...
    |
    end_subtask(subtask_id)
    |   |— Collect navigation events
    |   |— DOM snapshot at boundary

end_task()
|— Final DOM snapshot
|— Final navigation events
|— Tear down CDP session
|— Compute bundle hash (SHA256 of script content hashes)
|— Package SubtaskObservation
```

6.8 Framework Fingerprinting

Observer probes for frontend framework globals at task start:

Framework	Probes
React	__REACT_DEVTOOLS_GLOBAL_HOOK__, __REACT_QUERY_STATE__
Redux	__REDUX_DEVTOOLS_EXTENSION__, window.store.dispatch
Apollo	__APOLLO_CLIENT__
Vue	__VUE_DEVTOOLS_GLOBAL_HOOK__, __NUXT__
Angular	window.ng, window.Zone
Next.js	__NEXT_DATA__, script[src*="__next/"]

Plus a general enumeration of non-standard window properties (filtered against 200+ browser natives).

6.9 Bundle Hash Computation

At task end, the observer computes a **bundle hash** — a fingerprint of all JavaScript loaded

during the session:

```
def _compute_bundle_hash(self) -> str:
    content_hashes = sorted(s.content_hash for s in self._scripts.value)
    return hashlib.sha256(" | ".join(content_hashes).encode()).hexdigest()
```

This hash identifies the exact JavaScript bundle version. The executor uses it as a **precondition check** — if the site has deployed a new JS bundle since a tool was discovered, the tool's entry points may have changed and a canary test is triggered.

6.10 Script Deduplication

JavaScript sources captured via `Debugger.getScriptSource` are deduplicated by content hash (SHA256). This prevents the same library from being stored multiple times across page navigations.

7. Noise Filter — Traffic Cleaning

`noise_filter.py` (~210 lines) removes analytics, tracking, and ad traffic from captured HTTP requests. Without it, the learner would try to build tool graphs from Google Analytics calls.

7.1 Three-Tier Filtering

HTTP Request → Supplementary Domain Check → Adblock Engine → Domain Fal

Tier 1: Fast supplementary domain check — $O(1)$ frozenset lookup:

```
_SUPPLEMENTARY_NOISE_DOMAINS = frozenset({
    "firebase.googleapis.com",
    "firebaseinstallations.googleapis.com",
    "fonts.googleapis.com", "fonts.gstatic.com",
    "cdn.cookie law.org", "geolocation.onetrust.com",
    # ... 15 infrastructure/CDN/config domains
})
```

Tier 2: Brave adblock engine — full filter list matching:

```
engine = adblock.Engine(filter_set) # EasyPrivacy + EasyList
result = engine.check_network_urls(url, source_url, "xmlhttprequest")
return result.matched
```

Uses the Brave browser's open-source adblock library with two filter lists:

- **EasyPrivacy** — analytics, telemetry, tracking
- **EasyList** — ads and ad networks

Filter lists are downloaded once and cached for 7 days at `~/.cache/morphnet/adblock_engine.dat`.

Tier 3: Domain-only fallback — if the adblock library is unavailable:

```
def _domain_matches(hostname, domain_set):
    parts = hostname.split(".")
    for i in range(len(parts)):
        if ".".join(parts[i:]) in domain_set:
            return True
    return False
```

Checks against 64+ known tracking/analytics domains.

7.2 Noise Domain Lists

The observer also maintains its own noise domain list (20+ domains) for early filtering before requests even reach the noise filter:

```
_NOISE_DOMAINS = frozenset({
    "googletagmanager.com", "google-analytics.com",
    "segment.io", "mixpanel.com",
    "hotjar.com", "facebook.net",
    "clevertap.com", "doubleclick.net",
    "sentry.io", "newrelic.com",
    "fullstory.com", "clarity.ms",
    "amplitude.com", "heap.io",
    "intercom.io", "crisp.chat",
    # ... and more
})
```

7.3 Route Blocking

For Playwright's network idle detection, the noise filter provides a list of 64+ domains to block at the route level. This prevents analytics scripts from delaying page-ready detection:

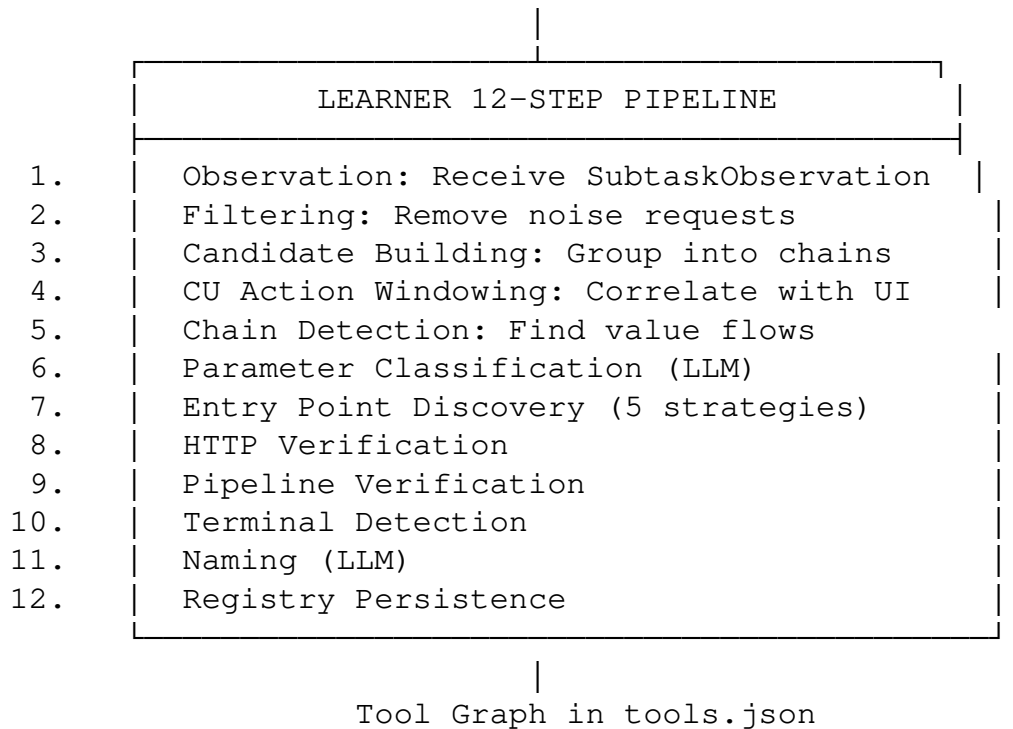
```
_ROUTE_BLOCK_DOMAINS = frozenset({
    # All supplementary + all major analytics/tracking
    "google-analytics.com", "googletagmanager.com",
    "segment.io", "mixpanel.com", "hotjar.com",
    "sentry.io", "newrelic.com", "datadog-agent.com",
    "fullstory.com", "clarity.ms", "amplitude.com",
    "cloudflareinsights.com", "bugsnag.com",
    # ... 64+ domains
})
```

8. Tool Discovery Pipeline — The Learner

`learner.py` is the heart of MorphNet's self-improvement mechanism. It takes raw HTTP traffic captured during CU execution and crystallizes patterns into deterministic, reusable **tool graphs**. This is a 12-step pipeline that transforms observed browser interactions into API-level automation.

8.1 The 12-Step Pipeline

CU executes subtask → Observer captures traffic



Step 1: Observation

The learner receives a `SubtaskObservation` from the observer containing all HTTP requests, CU actions, DOM snapshots, navigation events, and script sources captured during a subtask.

Step 2: Filtering (Two-Pass)

Traffic filtering uses two passes to separate signal from noise:

Pass 1 — Eliminate obvious noise:

Filter	What It Removes
Noise domains	Google Analytics, Mixpanel, Segment, Sentry, etc. (via <code>noise_filter</code>)
Preflight requests	<code>initiator_type</code> in <code>("preflight", "preload", "prefetch")</code>
OPTIONS requests	CORS preflight
Empty responses	<code>response_status == 0</code>
Trivial responses	Body < 100 bytes AND <code>keys ⊆ {"status", "ok", "success", "message"}</code>
Wrong content-type	Only keep if content-type contains <code>json</code> , <code>text</code> , <code>graphql</code> , <code>xml</code> , or <code>javascript</code> — OR body starts with <code>{</code> or <code>[</code>

Pass 2 — Keep requests whose data flows downstream:

For small responses (<100 bytes) that survived Pass 1, check if their data appears anywhere downstream:

- Any response leaf value (≥ 2 chars) appears in later request params → keep
- Any response leaf value appears in subsequent URL path/query changes → keep
- Response timestamp falls within a DOM content hash transition window → keep

Example: Filtering a train search session

Raw traffic captured: 47 requests

Pass 1 eliminates:

12 × google-analytics.com	(noise domain)
5 × googletagmanager.com	(noise domain)
3 × fonts.googleapis.com	(noise domain)
2 × sentry.io	(noise domain)
4 × OPTIONS preflight	(preflight)
3 × image/styleSheet resources	(wrong content-type)
2 × empty responses	(status 0)

→ 16 requests survive Pass 1

Pass 2 checks small responses:

4 config/health-check responses have no downstream flow → remove
 4 kept (leaf values appear in later request params)
 → 12 requests survive Pass 2

Final: 47 → 12 relevant API requests

Step 3: Candidate Building (Three Phases)

Phase 1 — CU Action Windowing:

CU actions are sorted by `timestamp_ms`. Each HTTP request is assigned to the most recent CU action that occurred before it:

Timeline:

t=0	CU: type [1] "Pune"	→ window 0
t=200	HTTP: GET /autocomplete?q=P	→ assigned to window 0
t=400	HTTP: GET /autocomplete?q=Pu	→ assigned to window 0
t=600	HTTP: GET /autocomplete?q=Pune	→ assigned to window 0
t=1200	CU: click [5] "Pune Junction"	→ window 1
t=1500	CU: type [2] "Mumbai"	→ window 2
t=1700	HTTP: GET /autocomplete?q=M	→ assigned to window 2
t=2500	CU: click [4] "Search Trains"	→ window 3
t=2800	HTTP: POST /search	→ assigned to window 3

Phase 2 — Group by (action_window, endpoint_fingerprint):

Requests with the same endpoint within the same action window are grouped. Each group = one semantic use of an endpoint.

Phase 3 — Prefix Chain Collapsing:

Keystroke progressions within a group are detected and collapsed:

```
Group: GET /autocomplete (window 0, 3 requests)
      q="P"      → q="Pu"      → q="Pune"
      ↑ prefix of next      ↑ most complete
```

Varying parameter: "q"

All values are prefixes of each other → This is a keystroke chain

Collapse: keep only the MOST COMPLETE request
→ GET /autocomplete?q=Pune

Result: 3 requests → 1 representative node

Core vs. Optional Parameters (computed across all representatives of same endpoint):

- `core_parameters` = $\cap$ (intersection) of all parameter sets — always present
- `optional_parameters` = $\cup$ (union) – core — sometimes present

Step 4: CU Action Windowing (Reasoning Capture)

Beyond grouping, the learner captures CU reasoning for each node:

```
HTTP: POST /api/trains/search (t=2800ms)
  └─ Nearest CU action: click [4] "Search Trains" (t=2500ms, Δ=300ms)
    └─ CU reasoning: "All fields filled. Click Search to find trains"
```

→ Node stores `cu_reasoning_sample` for later naming step

Step 5: Chain Detection

Find value flows between requests — where a response value from request A appears as a parameter in request B.

The `_TRIVIAL_VALUES` frozenset — excluded because they match coincidentally:

```
_TRIVIAL_VALUES = frozenset({
    "true", "false", "null", "none", "undefined",
    "0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "10",
    "yes", "no", "ok", "error", "success",
    "GET", "POST", "PUT", "DELETE",
    "asc", "desc", "ASC", "DESC", "DEFAULT",
})
```

Matching rules:

- Value must be ≥ 2 characters
- Value must NOT be in `_TRIVIAL_VALUES`
- Search only EARLIER nodes (strict temporal ordering)
- Exact string matching
- Returns ALL matching JSONPaths (a value may appear in multiple response fields)

Example: Chain detection in train search

```

Node n0: GET /autocomplete?q=Pune
Response: {"stationList": [
  {"stationCode": "PUNE", "stationName": "Pune Junction"},
  {"stationCode": "PNVL", "stationName": "Panvel"}
]}

Node n2: POST /search
Params: {from: "PUNE", to: "MUMBAI", date: "2026-05-01"}

Chain candidate found:
target: (n2, "from")
source: n0
matches: [{source_field: "$.stationList[0].stationCode", source_

No chain found for:
(n2, "date") = "2026-05-01" – no earlier node has this value
→ Will be classified as user_intent by Step 6

```

Step 6: Parameter Classification (LLM)

A single Gemini Flash call classifies every parameter across all nodes simultaneously:

Type	Description	Example
user_intent	Changes based on what the user wants to do	fromStation, toStation, date
chained	Extracted from a previous request's response	stationCode from autocomplete response
website_generated	Generated by the website (CSRF tokens, session IDs, timestamps)	_csrf, requestId, timestamp

Full input to the classification LLM:

```

{
  "task": "Search trains from Pune to Mumbai on May 1st",
  "actions_performed": "type [1] 'Pune'; click [5] 'Pune Junction'",
  "prior_graph_context": "Prior known workflows:\n- search_trains:",
  "nodes_summary": [
    {"id": "n0", "endpoint": "GET /api/stations/autocomplete", "method": "GET"},
    {"id": "n2", "endpoint": "POST /api/trains/search", "method": "POST"}
  ],
  "params_to_classify": [
    {
      "node_id": "n0",
      "param_name": "searchString",
      "param_value": "Pune",
      "data_type": "string",
      "is_required": true,
      "chain_candidates": [],
      "cu_action_context": "type [1] 'Pune' (From Station)",
      "profile_history": ["Pune", "Delhi", "Chennai"]
    }
  ]
}

```

```

    },
    {
      "node_id": "n2",
      "param_name": "fromStation",
      "param_value": "PUNE",
      "data_type": "string",
      "is_required": true,
      "chain_candidates": [{
        "source_node_id": "n0",
        "source_field": "$.stationList[0].stationCode",
        "source_value": "PUNE"
      }],
      "cu_action_context": "click [5] 'Pune Junction'",
      "profile_history": ["PUNE", "NDLS", "MAS"]
    },
    {
      "node_id": "n2",
      "param_name": "journeyDate",
      "param_value": "2026-05-01",
      "data_type": "date",
      "chain_candidates": [],
      "profile_history": ["2026-04-15", "2026-04-20"]
    }
  ]
}

```

Classification priority (from `classify_params.txt`):

1. First check chain candidates: clear semantic name match → **chained**
2. Then check user intent: semantic match from task/actions → **user_intent**
3. Everything else → **website_generated** (the default)

Key rules: Most parameters ARE `website_generated` (boolean configs, API keys, feature flags, pagination). Not all chain candidates are real chains (coincidental matches). Profile history helps: same value across tasks = likely `website_generated`; varies per task = likely `user_intent`.

LLM output:

```

{
  "classifications": [
    {
      "node_id": "n0",
      "param_name": "searchString",
      "role": "user_intent",
      "reasoning": "Varies per task, typed by user"
    },
    {
      "node_id": "n2",
      "param_name": "fromStation",
      "role": "chained",
      "chained_from_node_id": "n0",
      "chained_from_field": "$.stationList[0].stationCode",
      "reasoning": "stationCode from autocomplete feeds into search"
    },
    {
      "node_id": "n2",
      "param_name": "journeyDate",
      "role": "user_intent",
      "reasoning": "Date from task description, varies per task"
    }
  ]
}

```


Array edge detection: When a JSONPath contains an index like `[0]`, the learner creates a **selection edge**:

```
JSONPath: $.stationList[0].stationCode
          ↓ split
Array path: $.stationList           (the full array)
Item field: stationCode             (field to extract after LLM selection)
→ Edge: requires_selection=True, selection_array_path="$.stationList",
      selection_item_field="stationCode"
```

Step 7: Entry Point Discovery (5 Strategies)

For each HTTP request, the learner finds a way to invoke it programmatically. Strategies tried in preference order:

Strategy A — Reachable Global: Walk `window.*` up to depth 4 via CDP. Match function names against the request's initiator stack. Invocation: `await window.api.searchTrains({...})`.

Strategy B — Framework Dispatch: For Redux: find thunk in stack matching `fetch*/load*/search*/create*/update*/delete*` → `window.store.dispatch(window.fn({...}))`. For Apollo: `__APOLLO_CLIENT__.query()` or `.mutate()`.

Strategy C — Extracted IIFE: Extract function source from initiator stack. Skip library frames (node_modules, webpack, chunk-, vendor). Reject if contains `this./import/require/module`. (not self-contained). Wrap as IIFE.

Strategy D — DOM Replay: Find closest CU action before request (within 5s). For "type": `set el.value + dispatch input/change`. For "click": `dispatch MouseEvent`.

Strategy E — Fetch Replay (most common fallback):

```
await fetch('/api/search', {
  method: 'POST',
  headers: {"content-type": "application/json"},
  body: JSON.stringify({from: '${params.from}', to: '${params.to}'}),
  credentials: 'include' // Cookie forwarding
})
```

Example: Strategy preference chain

```
POST /api/trains/search
Initiator stack: [searchTrains (line 156), handleSubmit (line 89)]
```

```
Try A: Walk window.* → Found window.api.searchTrains ✓ → Use Strategy A
Try B (if A fails): No redux/apollo ✗
Try C: searchTrains() has "this." → not self-contained ✗
Try D: click [4] within 5s ✓ → acceptable but prefer fetch
Try E: Build fetch() → always available ✓ → Use Strategy E
```

Step 8: HTTP Verification

Execute the discovered entry point and verify it produces the expected HTTP request:

1. Check global path exists (for `cdp_eval_*` types):
`typeof window.api.searchTrains !== 'undefined' → ✓`
2. Re-invoke with example values:
→ Intercept HTTP via `page.on("request", ...)`
→ Wait 1.5s for async requests
3. Compare captured request to original:
→ `endpoint_fingerprints` must match
→ POST bodies: JSON key structure must match (sorted keys)
→ Exception: `dom_replay` may not produce HTTP – acceptable

Step 9: Pipeline Verification (End-to-End)

For multi-node graphs, verify the entire chain with fresh inputs:

1. Generate synthetic test task (LLM, `verification_task_gen.txt`):
"Find trains from Delhi to Agra on May 5th"
(Rules: DIFFERENT values from examples, realistic, all info inline)
2. Extract `user_intent` parameters (LLM):
{n0: {searchString: "Delhi"}, n1: {searchString: "Agra"},
n2: {journeyDate: "2026-05-05"}}
3. Execute full graph via executor:
n0: GET /autocomplete?searchString=Delhi → [NDLS, DLI, DSS...]
→ LLM selects: NDLS (New Delhi)
n1: GET /autocomplete?searchString=Agra → [AGC, AF, AGA...]
→ LLM selects: AGC (Agra Cantt)
n2: POST /search {from: NDLS, to: AGC, date: 2026-05-05}
→ {trains: [...]} ✓
4. Validate: every node returned data ✓, chains resolved ✓, selections

Step 10: Terminal Detection

A node is terminal if within **1-2 seconds** after its response:

- URL changed (1s window)
- DOM content hash changed (1s window)
- AXTree node count changed by > 20% (2s window)

n0 response → no URL/DOM change → not terminal

n2 response → URL → /results, DOM hash changed → TERMINAL ✓

Fallback: if none found → last node = terminal

Step 11: Naming (LLM)

A Gemini Flash call generates human-readable identifiers. The LLM receives node

descriptions (endpoint, CU reasoning, parameters), edge descriptions, parent graphs, and existing site graphs.

```
{
  "name": "search_trains",
  "description": "Search for trains between two stations. Requires city",
  "capability_statement": "Search for train availability between source",
  "node_descriptions": [
    {"node_id": "n0", "description": "Look up station code for departur"},
    {"node_id": "n1", "description": "Look up station code for destinat"},
    {"node_id": "n2", "description": "Search trains for given date and"}
  ]
}
```

Rules (from tool_naming.txt): snake_case action-oriented names. Bad: get_movies_mumbai (city in name), tool_v1.is_task_useful: FALSE for analytics/tracking/telemetry. cu_fallback_subtask: browser steps for when tool fails (NOT "use X tool").

Step 12: Registry Persistence

The completed graph is saved to morphnet/sites/{site_name}/tools.json after deduplication:

- **Exact match:** Graph with same identity hash already exists → skip
- **Subsumed:** New graph is a subset of existing → skip
- **Supergraph:** New graph extends existing → replace. Record existing as parent
- **Novel:** New graph → add to registry

8.6 Profile System

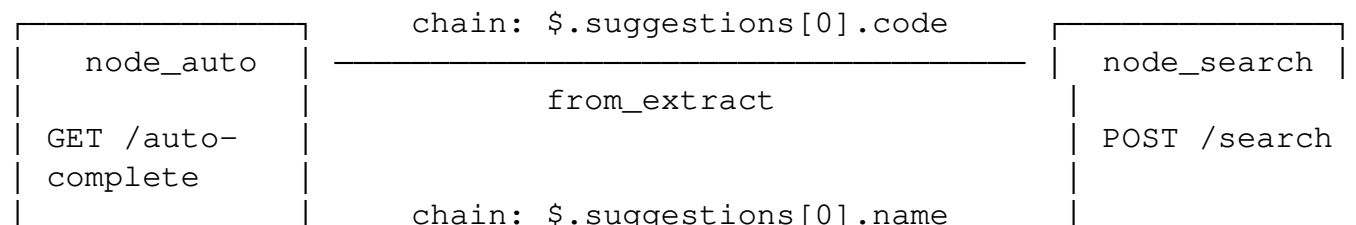
The learner maintains a **per-site parameter profile** that tracks historical values for each parameter across tasks:

```
# After each observation, profile is updated:
# SITES_DIR/{site}/profiles.json → {endpoint_fingerprint: {param_name:
profile["GET /api/autocomplete"]["searchString"] = ["Pune", "Delhi", "C"]
# Capped at 50 values per parameter
```

This profile data is passed to the parameter classification LLM as profile_history — helping distinguish user_intent (varies per task) from website_generated (same across tasks).

8.7 Graph Structure

A tool graph consists of nodes (HTTP requests) and edges (data flows):



Params:		Params:
q [user]	to_extract	from [chain]
_csrf [web]		to [user]
		date [user]
		_csrf [web]

GraphEdge fields:

- from_node_id, to_node_id — which nodes are connected
- from_extract — JSONPath to extract value from source response
- to_parameter — which parameter in the target to fill
- requires_selection — whether the user must choose from an array
- selection_array_path — JSONPath to the array
- selection_item_field — which field to display for selection

8.8 Raw tools.json — What the Learner Produces

Here is the actual `tools.json` for `ConfirmTkt`:

Registry file (`morphnet/sites/confirmtktk_com/tools.json`):

```
{
  "site": "confirmtktk_com",
  "graphs": [
    {
      "id": "8eba41e7d2ad46c2dc154bd5f8fce97a2deb76b3...",
      "name": "Search for trains between stations",
      "description": "This workflow allows a user to find available trains between two stations for a specific travel date.",
      "capability_statement": "Search for train availability and schedules between a source and destination station for a specific travel date.",
      "execution_stats": {"runs": 114, "successes": 114, "last_run": "2026-04-23T16:35:22"},
      "verified": true,
      "verification_only_read": true,
      "created_at": "2026-04-23T16:35:22",
      "file_path": "graphs/8eba41e7d2ad46c2dc154bd5f8fce97a2deb76b3...json"
    }
  ]
}
```

Processed for the planner (via `_get_available_graphs_summary()`):

```
Search for trains between stations [verified]
  Search for train availability and schedules between a source and destination station for a specific travel date.
  Stats: 114/114 successful executions
  Steps requiring input:
    n0 (Look up station code for departure): searchString
    n1 (Look up station code for destination): searchString
    n2 (Search trains for date): journeyDate
```

8.9 Raw profile.json — Learned Website Insights

Raw file (morphnet/sites/confirmtkkt_com/profile.json):

```
{
  "insights": [
    "The 'Search for trains between stations' executor tool is verified",
    "The executor API response provides a 'trainList' with 'avlClasses'",
    "Vande Bharat trains identifiable by train numbers (20661/20662)",
    "API data is comprehensive enough to answer most train queries",
    "The executor tool provides comprehensive fare data for all classes"
  ],
  "last_updated": 1714387200,
  "url": "https://www.confirmtkkt.com"
}
```

Processed for the planner (via _get_website_profile_summary()):

Website Profile:

- Verified executor provides API data: train names, classes, fares
- trainList has avlClasses/availabilityCache with precise fares
- Vande Bharat identified by numbers (20661) or classes (CC/EC)
- API sufficient for most queries – no UI navigation needed
- Fare data for all classes (SL, 3A, 2A, CC) in JSON response

The insights pipeline **appends** new learnings from each task run (up to 20 most recent), so knowledge accumulates across sessions.

8.10 Chaining Logic — Detailed Example

Consider a train search workflow where the user types "Pune" and the autocomplete returns a list of stations:

```
Node n0: GET /api/stations/autocomplete?searchString=Pune
```

```
Response:
```

```
{
  "stationList": [
    {"stationCode": "PUNE", "stationName": "Pune Junction"},
    {"stationCode": "PNVL", "stationName": "Panvel"},
    {"stationCode": "PURI", "stationName": "Puri"}
  ]
}
```

```
Chain: $.stationList[*].stationCode
       (requires_selection = true)
       (selection_array_path = "$.stationList")
       (selection_item_field = "stationCode")
```



```
Node n2: POST /api/trains/search
```

```

Body:
{
  "from": "PUNE",           ← chained from n0 (after LLM selection)
  "to": "MUMBAI",          ← chained from n1 (similar autocomplete)
  "date": "2026-05-01",    ← user_intent (from task description)
  "_csrf": "xK9m..."     ← website_generated (from cookie)
}

```

At execution time:

1. Node n0 executes → returns 3 stations
2. **Array selection** (LLM call): "User wants Pune. Options: [0] PUNE - Pune Junction, [1] PNVL - Panvel, [2] PURI - Puri. Select index 0."
3. Extract `stationCode` from selected item → "PUNE"
4. Substitute into n2's `from` parameter
5. Node n2 executes with resolved parameters

8.11 Validation Pipeline

After a graph is built, validation runs in two phases:

Phase 1: HTTP Verification (read-only graphs)

- Replace `${params.X}` with example values
- Intercept outgoing requests during execution
- Compare captured request fingerprint against original

Phase 2: Pipeline Verification (all multi-node graphs)

- Generate synthetic test task via LLM (e.g., "Find trains from Delhi to Agra on May 5th")
- Extract `user_intent` parameters from task
- Execute full graph end-to-end via executor
- Validate: every node returned data, every chain resolved, every selection worked

9. Manifest — Tool Registry & Graph Identity

`manifest.py` manages the tool registry — storing, finding, and deduplicating tool graphs.

9.1 Finding Candidate Graphs

When the orchestrator wants to route a subtask to a tool, `find_candidates()` searches the registry:

```

def find_candidates(site, subtask_description, page_url, embedding_client):
    # 1. Load all graphs for the site
    all_graphs = list_graphs(site)

    # 2. Filter by URL precondition (fnmatch)

```

```

candidates = [g for g in all_graphs
               if fnmatch(page_url, g.preconditions.get("url_pattern"))]

# 3. Rank by semantic similarity (if embeddings available)
if embedding_client is not None:
    embeddings = load_embeddings(site) # {graph_id: [float, ...]}
    if embeddings:
        candidates = _rank_by_similarity(candidates, subtask_description,
                                         embeddings, embedding_client)

return candidates

```

Embedding-based ranking:

```

def _rank_by_similarity(candidates, query, embeddings, embedding_client):
    query_embedding = embedding_client.embed(query) # Embed the subtask

    scored = []
    for graph in candidates:
        graph_emb = embeddings.get(graph.id)
        if graph_emb is None:
            scored.append((graph, 0.0)) # No embedding → lowest rank
            continue
        sim = _cosine_similarity(query_embedding, graph_emb)
        scored.append((graph, sim))

    scored.sort(key=lambda x: x[1], reverse=True)
    return [g for g, _ in scored]

def _cosine_similarity(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = sum(x * x for x in a) ** 0.5
    norm_b = sum(x * x for x in b) ** 0.5
    return dot / (norm_a * norm_b) if norm_a and norm_b else 0.0

```

Embeddings are stored in `morphnet/sites/{site}/embeddings.json` as `{graph_id: [float, ...]}`. Computed from the graph's name, description, and capability statement.

Example: Candidate selection for "Find cheapest train from Pune to Mumbai"

```

Site: confirmkt_com
Available graphs:
  search_trains [verified] - similarity: 0.87 ← BEST MATCH
  check_pnr [probationary] - similarity: 0.23
  get_insurance [unverified] - similarity: 0.11

URL filter: all pass (url_pattern = "")
Result: [search_trains, check_pnr, get_insurance] (ranked by similarity)

```

9.2 Graph Data Structures

```

@dataclass
class GraphNode:
    id: str # Stable within graph (e.g., "
    endpoint_fingerprint: str # Canonical form: "POST /api/t
    http_method: str # "GET", "POST", etc.
    url_template: str # With {param} placeholders
    request_type: str # "rest", "graphql", "json_rpc
    core_parameters: list[ParameterSpec] # Always present
    optional_parameters: list[ParameterSpec]
    response_schema: dict # Inferred JSON schema of resp
    response_extract_paths: dict # name → JSONPath for chained
    invocation: NodeInvocation # How to execute (strategy + e
    cu_reasoning_sample: str # CU's reasoning when this nod
    node_description: str # Human-readable role

@dataclass
class ParameterSpec:
    name: str
    role: str # "user_intent", "chained", "w
    value_example: str # Observed value from discover
    data_type: str # "string", "integer", "boolea
    is_required: bool
    chained_from: str | None # "source_node.jsonpath" if ch
    transform: str | None # "uppercase", "lowercase", "d

@dataclass
class GraphEdge:
    from_node_id: str
    to_node_id: str
    from_extract: str # JSONPath from source respons
    to_parameter: str # Parameter name in target nod
    requires_selection: bool = False # Array source needs LLM pick
    selection_array_path: str = "" # JSONPath to array (without i
    selection_item_field: str = "" # Field to extract from select

```

9.3 Graph Identity

Each graph has a deterministic identity based on its structure:

```

# SHA256 of sorted (endpoint fingerprints + edge tuples)
identity = sha256(
    sorted([node.endpoint_fingerprint for node in graph.nodes]) +
    sorted([(e.from_node, e.to_node, e.from_extract, e.to_param) for e
])

```

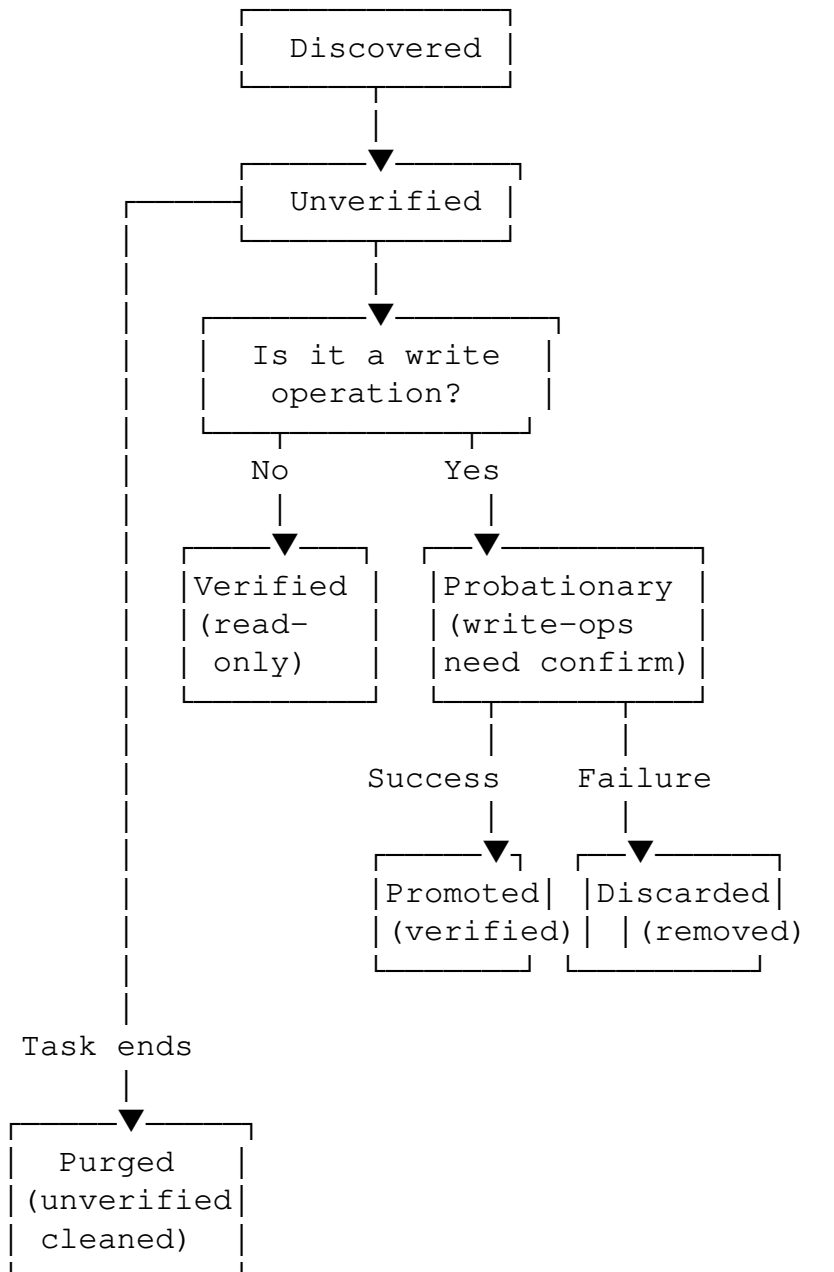
This enables deduplication: if the learner discovers the same API chain twice, the identity hash matches and the duplicate is skipped.

9.4 Registry Deduplication

When a new graph is registered:

1. **Exact match** (same identity hash) → skip
2. **Subsumed** (new graph's nodes/edges are a subset of existing) → skip
3. **Supergraph** (new graph extends existing with more nodes) → replace existing
4. **Novel** → add as new graph

9.5 Tool Lifecycle



10. Executor — Graph Execution Engine

`executor.py` executes tool graphs by sending HTTP requests via CDP and chaining responses between nodes.

10.1 Execution Flow

```

async def execute(graph, user_intent_params, session):
    # 1. Topological sort of nodes
    ordered = topological_sort(graph.nodes, graph.edges)

    node_outputs = {}

    for node in ordered:
        # 2. Resolve chained parameters from upstream outputs
        params = resolve_chains(node, graph.edges, node_outputs)

        # 3. Fill user_intent parameters
        params.update(user_intent_params.get(node.id, {}))

        # 4. Fill website_generated parameters (cookies, CSRF, etc.)
        params.update(await extract_website_params(node, session))

        # 5. Execute via CDP
        response = await execute_node_via_cdp(node, params, session)

        # 6. Handle array selection (if edge requires it)
        if edge.requires_selection and isinstance(response[array_path], su
            selected = await select_from_array(response[array_path], su

        # 7. Store output for downstream nodes
        node_outputs[node.id] = response

    return ExecutionResult(status="success", node_outputs=node_outputs)

```

10.2 Node Execution via CDP

Each node is executed as a JavaScript `fetch()` call inside the browser:

```

# Wrapped in async IIFE for CDP evaluation
js = f"""
(async () => {{
    const response = await fetch('{node.url}', {{
        method: '{node.method}',
        headers: {json.dumps(node.headers)},
        body: {json.dumps(body)},
        credentials: 'include'
    }});
    return await response.json();
}})()
"""
result = await session.page.evaluate(js)

```

This runs inside the browser context, so cookies and authentication state are automatically included.

10.3 Chain Resolution

When a node has chained parameters, values are extracted from upstream outputs via JSONPath:

```
# Edge: from_node="node_auto", from_extract="$.suggestions[0].code", to
upstream_output = node_outputs["node_auto"]
value = jsonpath_extract(upstream_output, "$.suggestions[0].code")
# value = "PUNE"
params["from"] = value
```

10.4 Array Selection

When a chained value points to an array, the executor must select the right item:

```
# Single item → return immediately
if len(array) == 1:
    return array[0]

# Multiple items → LLM selects
# Items capped at 20, dict items show first 8 fields
selected = await call_gemini(
    model="gemini-3-flash-preview",
    contents=[f"Select the best match for: {subtask}\n\nOptions:\n{form
    response_schema={"selected_index": int, "reasoning": str, "confiden
)
return array[selected["selected_index"]]
```

10.5 Parameter Types and Resolution

Type	Source	How Filled
user_intent	Subtask description	LLM extracts from subtask text (via intent_extraction.txt prompt)
chained	Upstream node response	JSONPath extraction + optional LLM array selection
website_generated	Browser state	Uses param.value_example (stored from discovery time)

Intent extraction — The LLM receives the subtask, graph capability, and parameter definitions:

```
Prompt (intent_extraction.txt):
Task: "Search trains from Pune to Mumbai on May 1st"
Workflow: search_trains - "Search for train availability..."
Parameters:
  n0.searchString: "Station code for departure" (example: "Pune")
  n1.searchString: "Station code for destination" (example: "Mumbai")
  n2.journeyDate: "Travel date" (example: "2026-04-15")

Rules:
- Same-named params on different nodes need different values
- Match format of example value (e.g., DD-MM-YYYY for dates)
```

- Set null if cannot extract confidently

Output:

```
{n0.searchString: "Pune", n1.searchString: "Mumbai", n2.journeyDate:
```

10.6 Parameter Transformations

Parameters can have transformations applied before injection:

```
def _apply_transform(value, transform):
    if transform == "uppercase":    return value.upper()
    if transform == "lowercase":    return value.lower()
    if transform == "date_iso_to_dmy":
        # YYYY-MM-DD → DD-MM-YYYY
        parts = value.split("-")
        return f"{parts[2]}-{parts[1]}-{parts[0]}"
    if transform == "url_encode":    return urllib.parse.quote(value)
    return value
```

Example: Task says "May 1st 2026" → LLM extracts "2026-05-01" → transform date_iso_to_dmy → API receives "01-05-2026".

10.7 Precondition Checks

Before executing a graph, the executor runs precondition checks:

1. URL pattern: `fnmatch(current_url, graph.preconditions["url_pattern"])`
2. Required globals: `typeof window.api !== 'undefined'` (for `cdp_eval_*`)
3. Bundle hash: if site's JS bundle changed since discovery → run canary

Canary test: Execute the simplest node with its example values. Validate that the response structure matches the learned schema. If structure changed → mark graph as "degraded" instead of executing.

10.8 ExecutionResult

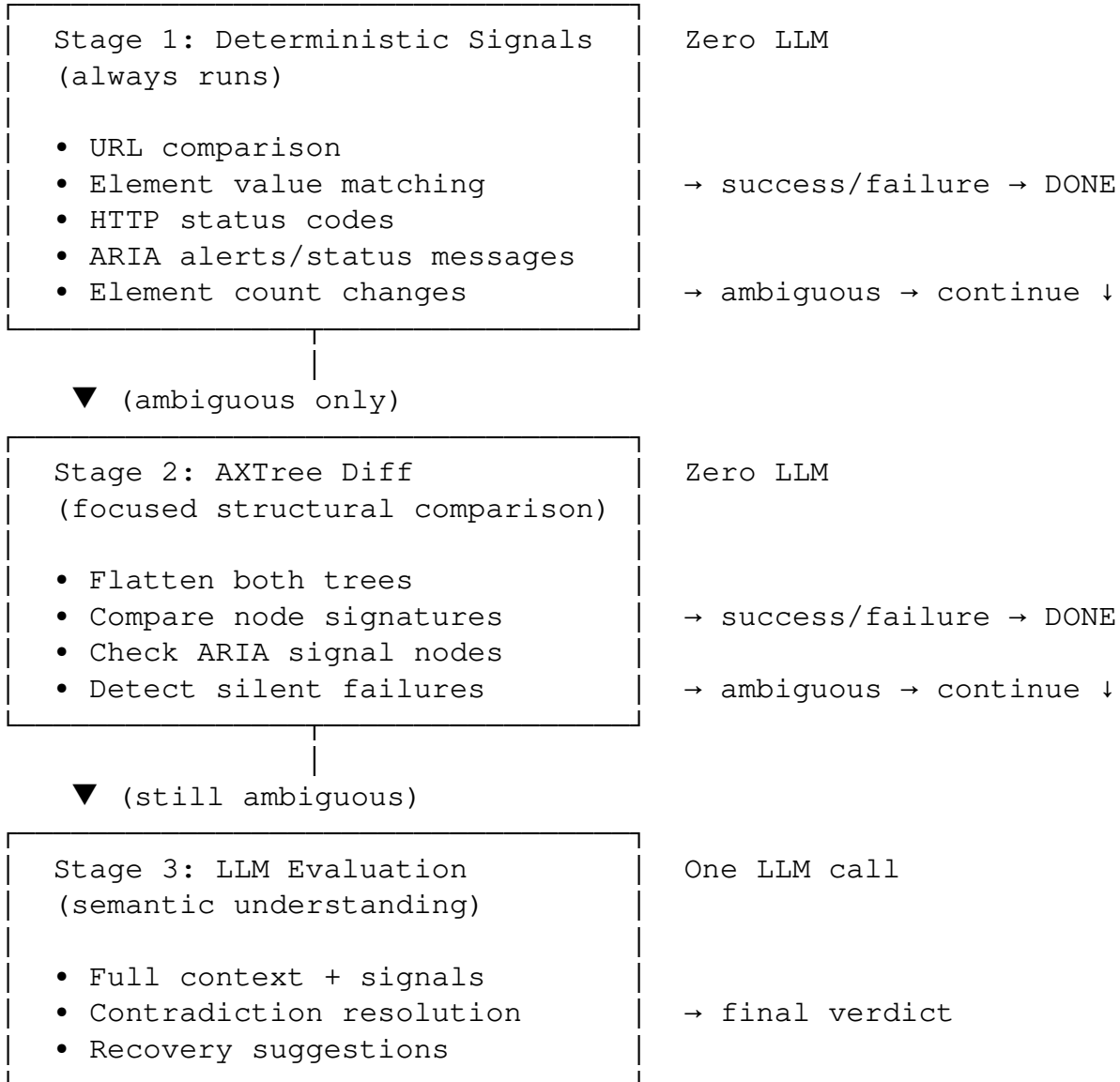
```
@dataclass
class ExecutionResult:
    status: str          # "success", "not_applicable", "degraded",
                        # "execution_error", "completion_timeout"
    result: dict | None
    failed_node_id: str | None
    reason: str | None
    node_outputs: dict    # {node_id: response_data}
```

11. Reflector — Verification Pipeline

`reflector.py` (~1350 lines) implements a three-stage verification pipeline. The key principle: **avoid LLM calls when deterministic signals suffice**. Only ~2-3 LLM calls happen per subtask.

11.1 Three-Stage Architecture

Action executed



11.2 Deterministic Signals (Stage 1)

The DeterministicSignals dataclass — collected automatically after every action (zero LLM):

```
@dataclass
class DeterministicSignals:
    url_before: str
    url_after: str
    url_changed: bool
    target_element_value_before: str | None
    target_element_value_after: str | None
    expected_value: str | None
    value_matches_expected: bool | None
```

```

http_status_codes: list[int]
has_error_status: bool # Any 4xx/5xx
has_success_status: bool # Any 2xx state-changing request
aria_alerts: list[str] # role="alert" or "alertdialog"
aria_status_messages: list[str] # role="status" or aria-live="p
aria_invalid_fields: list[dict] # {element_id, field_name} with
new_dialogs: list[str] # New role="dialog" text
elements_added: int
elements_removed: int
deterministic_verdict: str # "success" | "failure" | "ambi
verdict_reason: str

```

Verdict rules by action type:

type / select:

- **SUCCESS:** `value_matches_expected == True` → "Value set to '{val}' as expected"
- **FAILURE:** `value_matches_expected == False` → "Expected '{expected}' but got '{actual}'"
- **FAILURE:** `has_error_status` → "HTTP error during action"
- **AMBIGUOUS:** otherwise (couldn't verify value)

navigate / go_back:

- **SUCCESS:** `url_changed` → "URL changed to {url_after}"
- **FAILURE:** `url_after == url_before` → "URL did not change"

scroll (CRITICAL — hard-coded, no LLM override):

- **SUCCESS:** `elements_added > 0` → "{N} new elements revealed"
- **FAILURE:** `elements_added == 0` → "Scroll produced no new elements"

This is deterministic and Stage 3 CANNOT override it. The LLM was overriding scroll failures to "success" ~96% of the time, causing infinite scroll loops on LEGO, BMS, and ConfirmTkt.

click:

- **FAILURE:** `aria_alerts present` → "Alert: {alert_text}"
- **FAILURE:** `has_error_status` → "HTTP error: {status_codes}"
- **SUCCESS:** `url_changed` → "Navigated to {url_after}"
- **SUCCESS:** `has_success_status` → "Successful state-changing request"
- **SUCCESS:** `elements_added > 0` or `elements_removed > 0` → "Page changed: +{added}/-{removed}"
- **AMBIGUOUS:** otherwise
- **Note:** `aria_invalid` NOT checked for clicks — only for type/select

press_key (Enter):

- **SUCCESS:** `has_success_status AND url_changed` → "Form submitted — URL changed"
- **SUCCESS:** `url_changed` → "Enter — navigated to {url_after}"
- **SUCCESS:** `has_success_status` → "Enter — successful response"

- FAILURE: `has_error_status` → "HTTP error after Enter"
- FAILURE: `aria_alerts` → "Alert after Enter"
- SUCCESS: `elements_added > 0` or `elements_removed > 0` → "Enter — page changed"
- AMBIGUOUS: otherwise

press_key (Tab/Escape/ArrowDown/ArrowUp/Space): Always SUCCESS.

hover: FAILURE if Playwright failed; SUCCESS otherwise.

note: Always SUCCESS.

wait: SUCCESS if elements changed; AMBIGUOUS if no change.

11.3 AXTree Diff (Stage 2)

When Stage 1 returns "ambiguous", the reflector computes a focused diff between before/after AXTree snapshots:

```
def compute_axtree_diff(before, after):
    # Flatten both trees to (role, name, path) tuples
    before_flat = flatten_axtree(before)
    after_flat = flatten_axtree(after)

    # Detect changes
    added = after_sigs - before_sigs          # New elements
    removed = before_sigs - after_sigs         # Removed elements
    text_changed = ...                        # Same position, different text
    prop_changed = ...                        # Same sig, different properties
```

Example AXTree diff:

Before (train search form):

```
heading "Train Search"
textbox "From Station" = ""
textbox "To Station" = ""
button "Search"
```

After (results loaded):

```
heading "Train Search"
textbox "From Station" = "PUNE"
textbox "To Station" = "MUMBAI"
button "Search"
heading "15 Trains Found"          ← ADDED (priority HIGH)
listitem "Pune Intercity ₹350"     ← ADDED (priority MEDIUM)
listitem "Deccan Express ₹280"    ← ADDED (priority MEDIUM)
```

Diff output:

```
[HIGH]  ADDED: heading "15 Trains Found"
[HIGH]  CHANGED: textbox "From Station" → value: "" → "PUNE"
[HIGH]  CHANGED: textbox "To Station" → value: "" → "MUMBAI"
[MED]   ADDED: listitem "Pune Intercity ₹350"
```

```
[MED]    ADDED: listitem "Deccan Express ₹280"
```

Silent failure detection: If a submit action produces zero changes and no ARIA alerts, the reflector injects a warning. Submit actions are detected structurally:

- Click on element with `type="submit"`, `formaction`, or `formmethod` attribute
- Click on `role="button"` inside a `<form>` ancestor
- `press_key` with text `"enter"`

WARNING: A submit action was performed but no page changes were detected. This may indicate a silent failure – the form may not have actually submitted.

Example: Silent failure detection

```
Action: click [4] "Submit Order"
```

```
Element: <button type="submit"> inside <form>
```

```
AXTree diff: "NO MEANINGFUL CHANGES DETECTED"
```

```
ARIA alerts: none
```

```
HTTP errors: none
```

→ Silent failure detected: submit + zero changes + no errors

→ Verdict: FAILURE with correction hint:

"Form may not have submitted. Check if required fields are empty if JavaScript validation prevented the submit."

11.4 Subtask Reflection

At the end of each subtask, a deep reflection uses Gemini Pro Preview:

```
SUBTASK_REFLECTION_SCHEMA = {
    "subtask_achieved": bool,          # Did we complete the subtask?
    "confidence": float,               # 0.0 to 1.0
    "outcome_summary": str,            # What happened (2-3 sentences)
    "failure_analysis": str,           # Root cause if failed
    "recommendation": enum,            # proceed/retry_same/retry_diff
                                        # prune_branch/task_impossible
    "page_state_summary": str,         # Current page state
    "extracted_data": str,             # Data found for retrieval task
    "false_positive_check": str,       # Did CU claim success without
    "reasoning": str,
    "evidence_sources": list[str],
}
```

The reflection prompt includes:

- Subtask description
- Condensed action log (per-action summaries)
- Current page AXTree (reflector view — chrome-compressed, card-aware)
- **Focused DOM excerpt** (see below)
- All ARIA signals collected across actions (deduplicated)
- Agent notes

Focused DOM extraction — Instead of the full DOM (which would be too large), the reflector extracts ~100 lines around verification-relevant keywords:

```
# Keywords searched in DOM:
["form", "alert", "status", "error", "success", "warning",
 "cart", "basket", "total", "flash", "notification", "message",
 "data-count", "data-cart", "data-qty", "aria-live"]

# For each keyword match: capture 3 lines before + match + 3 lines after
# Deduplicate, return first 100 lines
```

False positive check — A required field in subtask reflection:

```
"false_positive_check": "Did the CU agent claim success without perform
the key action? Report any 'claimed but not executed' patterns."
```

This catches a common failure: the agent clicks a button and reports success, but the button didn't actually trigger the expected behavior (e.g., a disabled button, a loading spinner that blocked the click).

11.5 Failure Types

none	# Success
wrong_element	# Clicked/typed on wrong element
element_not_found	# Element ID invalid or gone
value_not_set	# Type action didn't set value
navigation_unexpected	# Navigated to wrong URL
no_visible_change	# Action executed but no visible effect
error_message_appeared	# Alert/error dialog appeared
form_validation_failed	# Form field marked invalid
server_error	# HTTP 4xx/5xx
claimed_but_not_executed	# CU claimed success without performing action
page_not_ready	# Page still loading
unknown	# Unclassifiable failure

11.6 MCP Call Reflection

When the executor runs a tool graph, reflection follows a different path:

Stage	Check	Outcome
A	HTTP status codes + GraphQL errors	success if 2xx, failure if errors
B	Response structure vs learned template	degraded if structure changed
C	Page state AXTree diff	Confirms side effects (write ops)

Recommendations: proceed / retry_mcp / fallback_to_cu / mark_mcp_degraded

12. Trace — Deterministic JSONL Logging

`trace.py` (~380 lines) is a deterministic JSONL recorder. **Zero LLM calls** — all trace data comes FROM Gemini schema fields (`reasoning`, `evidence_sources`, `confidence`), never from post-hoc parsing.

12.1 Core Data Structure

```
@dataclass
class TraceEntry:
    timestamp: float           # Unix timestamp
    trace_id: str              # Unique 12-char hex ID
    parent_id: str | None      # Links to parent span (hierarchical)
    module: str                # "cu_agent", "orchestrator", "reflec"
    event_type: str            # Semantic event name
    summary: str               # Human-readable one-liner
    detail: dict               # Arbitrary structured data
    reasoning: str | None      # Direct from Gemini schema
    evidence: list[Evidence]    # What informed the decision
    outcome: str | None        # "success" or "failure"
    error: str | None          # Exception message
    duration_ms: float | None  # Wall-clock time
    confidence: float | None   # Direct from Gemini schema (0.0-1.0)

@dataclass
class Evidence:
    source: str                # "dom", "axtree", "screenshot", "traffic", "m
    description: str           # Human-readable explanation
    element_id: int | None
    raw_excerpt: str | None    # First 500 chars
```

12.2 Logging API

Simple event logging:

```
trace.log("orchestrator", "task_started", "Task: Find cheapest train...
        detail={"url": "https://confirmtkkt.com", "max_subtasks": 15})
```

Timed span with incremental evidence:

```
with trace.span("cu_agent", "action_selected", "Step 5: click [4]") as
    s.add_evidence(Evidence("axtree", "Element [4] is Search button"))
    s.set_reasoning(action["reasoning"])      # Direct from Gemini
    s.set_confidence(action["confidence"])    # Direct from Gemini
    for src in action["evidence_sources"]:    # Direct from Gemini
        s.add_evidence(Evidence("model_output", src))
    s.set_outcome("success")
# Duration automatically measured on span exit
```

12.3 Event Types by Module

Module	Event Type	When
--------	------------	------

orchestrator	task_started	Task begins
orchestrator	plan_decision	Planner LLM called (span)
orchestrator	tree_branch	Planning tree branched
orchestrator	tree_prune	Planning tree pruned
orchestrator	executor_success	Executor completed subtask
orchestrator	executor_fallback_to_cu	Executor failed, CU fallback
orchestrator	graph_promoted	Probationary → verified
orchestrator	graph_discarded	Probationary → removed
orchestrator	task_completed	Task finished
orchestrator	task_budget_exhausted	Max subtasks reached
cu_agent	action_selected	CU selects action (span)
cu_agent	subtask_started	CU begins subtask
cu_agent	subtask_completed	CU finishes subtask
cu_agent	note_recorded	CU records observation
reflector	deterministic_signals	Stage 1 verdict
reflector	axtree_diff	Stage 2 diff result
reflector	llm_action_eval	Stage 3 LLM verdict (span)
reflector	subtask_reflection	End-of-subtask LLM eval (span)
session_manager	session_started	Browser connected
session_manager	action_executed	Browser action completed
session_manager	popup_dismissed	Modal auto-dismissed
session_manager	screenshot_taken	Screenshot captured

12.4 Complete Trace Entry Example

```
{
  "timestamp": 1714427422.123,
  "trace_id": "a1b2c3d4e5f6",
  "parent_id": null,
  "module": "cu_agent",
  "event_type": "action_selected",
  "summary": "Step 5: click [4] Search Trains",
  "detail": {
    "action_type": "click",
    "element_id": 4,
    "action": {
      "action_type": "click",
      "element_id": 4,
      "reasoning": "All fields are filled. Click Search to find t",
      "confidence": 0.95,
      "evidence_sources": ["[4] btn'Search Trains' is the submit b",
                           "[1] txt'From'='PUNE' is filled",
                           "[2] txt'To'='MUMBAI' is filled"]
    }
  }
}
```

```

    },
    "reasoning": "All fields are filled. Click Search to find trains",
    "evidence": [
      {"source": "model_output", "description": "[4] btn'Search Train'"},
      {"source": "model_output", "description": "[1] txt'From'='PUNE'"},
    ],
    "outcome": "success",
    "error": null,
    "duration_ms": 2847.34,
    "confidence": 0.95
  }
}

```

12.5 Output Directory Structure

```

results/2026-04-29_093022_123/
├── trace.jsonl                # All events, one per line
├── planning_tree.mermaid      # Mermaid visualization of planning
├── steps/
│   ├── plan_001.json          # Step 1: raw AXTree + processed view
│   ├── plan_001_screenshot.jpg # Step 1 screenshot
│   ├── plan_002.json
│   ├── cu_step_001.json       # CU action details
│   └── ...
├── prompt_made/
│   └── *.txt                  # Raw prompts sent to Gemini (for de

```

Entries are flushed immediately to disk (`file.flush()` after every write) — crash-safe recording.

13. End-to-End Walkthrough

Let's trace a complete task: **"Find the cheapest train from Pune to Mumbai on May 1st"** on ConfirmTkt.

Step 1: Session Start

```

session_manager:
├── Launch Chrome with stealth flags (port 9222)
├── Connect via CDP
├── Apply 3-layer stealth (playwright-stealth + custom + UA matching)
├── Navigate to https://www.confirmktkt.com
├── Dismiss cookie consent popup
├── Sync cookies to curl_cffi session
└── site_name = "confirmktkt_com"
    └── Load profile.json (prior insights), tools.json (learned grap

```

Step 2: Orchestrator Loop — Iteration 1

```

orchestrator:

```

- └─ Get page state
 - └─ AXTree: 342 nodes, 24 buttons, 5 textboxes
 - └─ Interactive elements: 45 (filtered to viewport)
 - └─ DOM: 85KB cleaned HTML
- └─ Build representations
 - └─ Orchestrator view (text-only, no IDs): "Search form, popular
 - └─ DOM summary: "Form with from/to/date fields"
 - └─ Planning tree: "plan_0 → plan_0_1 (Initial approach) ← CURREN
 - └─ Tool summary: "search_trains [verified] – 15/18 success"
 - └─ Profile: "Date picker mandatory, station autocomplete require
- └─ Call planner (Gemini Flash, temp 0.4)
 - └─ Response:
 - planning_action: "continue"
 - routing: "executor"
 - graph_name: "search_trains"
 - next_subtask: "Search trains from Pune to Mumbai on May 1st"
- └─ Route to executor
 - └─ find_candidates("search_trains") → matched
 - └─ Extract user_intent: {from: "PUNE", to: "MUMBAI", date: "2026
 - └─ Execute graph:
 - node_auto: GET /autocomplete?q=PUNE → {suggestions: [{code: "
 - node_search: POST /search {from: "PUNE", to: "MUMBAI", date:
 - {trains: [{name: "Pune Intercity", price: 350}, ...]}
 - └─ Result: success
 - └─ Response summary: 15 trains, prices ₹280–₹1200

Step 3: Orchestrator Loop — Iteration 2

orchestrator:

- └─ Executor response injected into planning prompt:
 - "Executor API Response Data:
 - [{name: 'Pune Intercity', price: 350}, {name: 'Deccan Express',
 - If the task's answer is in this data, use complete_task."
- └─ Call planner
 - └─ Response:
 - planning_action: "complete_task"
 - final_answer: "Cheapest: Deccan Express (11007) at ₹280, depa
 - task_success: true
- └─ Return TaskResult(success=true, final_answer="Deccan Express at ₹

Total: 2 orchestrator iterations, 1 executor call, 0 CU actions. The learned tool graph handled everything via API.

If no tool graph existed, the system would have used CU to:

1. Fill the search form (3-5 actions)

2. Read results (1-2 scroll actions)
3. Extract the cheapest train (1 note action)

And the observer/learner would have captured the HTTP traffic to build the `search_trains` graph for next time.

This document covers MorphNet's complete architecture. For code-level details, refer to the source files with the line numbers referenced throughout.